

云原生训练营

模块七：Kubernetes 控制平面组件

孟凡杰

前 eBay 资深架构师



目录

- 调度
- Controller Manager
- kubelet
- CRI
- CNI
- CSI

1. 调度

kube-scheduler



kube-scheduler 负责分配调度 Pod 到集群内的节点上，它监听 kube-apiserver，查询还未分配 Node 的 Pod，然后根据调度策略为这些 Pod 分配节点（更新 Pod 的 nodeName 字段）。

调度器需要充分考虑诸多的因素：

- 公平调度；

- 资源高效利用；

- QoS；

- affinity 和 anti-affinity；

- 数据本地化（data locality）；

- 内部负载干扰（inter-workload interference）；

- deadlines。

调度器

kube-scheduler 调度分为两个阶段，predicate 和 priority:

- predicate: 过滤不符合条件的节点;
- priority: 优先级排序，选择优先级最高的节点。

Predicates 策略

PodFitsHostPorts: 检查是否有 Host Ports 冲突。

PodFitsPorts: 同 PodFitsHostPorts。

PodFitsResources: 检查 Node 的资源是否充足，包括允许的 Pod 数量、CPU、内存、GPU 个数以及其他的OpaqueIntResources。

HostName: 检查 pod.Spec.NodeName 是否与候选节点一致。

MatchNodeSelector: 检查候选节点的 pod.Spec.NodeSelector 是否匹配。

NoVolumeZoneConflict: 检查 volume zone 是否冲突。

Predicates 策略

MatchInterPodAffinity: 检查是否匹配 Pod 的亲合性要求。

NoDiskConflict: 检查是否存在 Volume 冲突，仅限于 GCE PD、AWS EBS、Ceph RBD 以及 iSCSI。

PodToleratesNodeTaints: 检查 Pod 是否容忍 Node Taints。

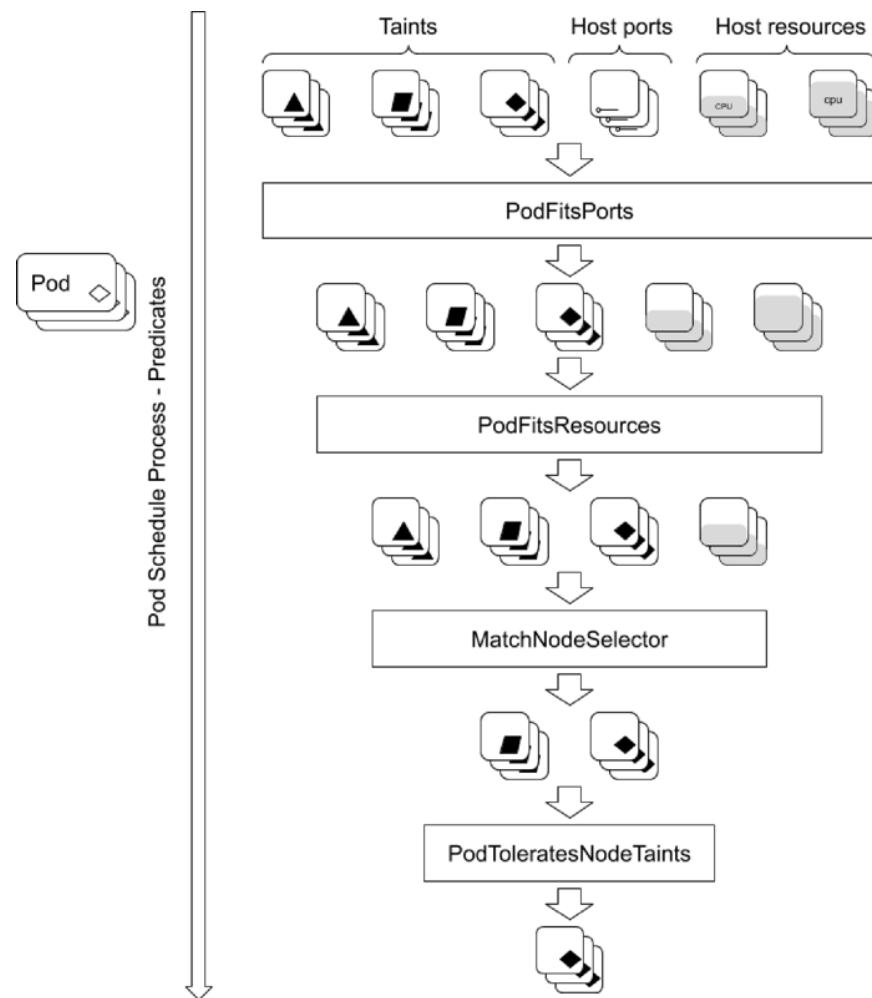
CheckNodeMemoryPressure: 检查 Pod 是否可以调度到 MemoryPressure 的节点上。

CheckNodeDiskPressure: 检查 Pod 是否可以调度到 DiskPressure 的节点上。

NoVolumeNodeConflict: 检查节点是否满足 Pod 所引用的 Volume 的条件。

还有很多其他策略，你也可以编写自己的策略。

Predicates plugin工作原理



Priorities 策略

SelectorSpreadPriority: 优先减少节点上属于同一个 Service 或 Replication Controller 的 Pod 数量。

InterPodAffinityPriority: 优先将 Pod 调度到相同的拓扑上（如同一个节点、Rack、Zone 等）。

LeastRequestedPriority: 优先调度到请求资源少的节点上。

BalancedResourceAllocation: 优先平衡各节点的资源使用。

NodePreferAvoidPodsPriority: [alpha.kubernetes.io/preferAvoidPods](https://kubernetes.io/docs/reference/kubernetes-api/object-models/pod/#pod-scheduling) 字段判断,权重为 10000, 避免其他优先级策略的影响。

Priorities 策略

NodeAffinityPriority: 优先调度到匹配NodeAffinity的节点上。

TaintTolerationPriority: 优先调度到匹配TaintToleration的节点上。

ServiceSpreadingPriority: 尽量将同一个service的Pod分布到不同节点上，已经被SelectorSpreadPriority替代（默认未使用）。

EqualPriority: 将所有节点的优先级设置为1（默认未使用）。

ImageLocalityPriority: 尽量将使用大镜像的容器调度到已经下拉了该镜像的节点上（默认未使用）。

MostRequestedPriority: 尽量调度到已经使用过的Node上，特别适用于cluster-autoscaler（默认未使用）。

资源需求

- CPU
 - requests
 - Kubernetes调度Pod时，会判断当前节点正在运行的Pod的CPU Request的总和，再加上当前调度Pod的CPU request，计算其是否超过节点的CPU的可分配资源
 - limits
 - 配置cgroup以限制资源上限
- 内存
 - requests
 - 判断节点的剩余内存是否满足Pod的内存请求量，以确定是否可以将Pod调度到该节点
 - limits
 - 配置cgroup以限制资源上限

磁盘资源需求

容器临时存储（ephemeral storage）包含日志和可写层数据，可以通过定义Pod Spec中的limits.ephemeral-storage和requests.ephemeral-storage来申请。

Pod调度完成后，计算节点对临时存储的限制不是基于CGroup的，而是由kubelet定时获取容器的日志和容器可写层的磁盘使用情况，如果超过限制，则会对Pod进行驱逐。

Init Container的资源需求

- 当kube-scheduler调度带有多个init容器的Pod时，只计算cpu.request最多的init容器，而不是计算所有的init容器总和。
- 由于多个init容器按顺序执行，并且执行完成立即退出，所以申请最多的资源init容器中的所需资源，即可满足所有init容器需求。
- kube-scheduler在计算该节点被占用的资源时，init容器的资源依然会被纳入计算。因为init容器在特定情况下可能会被再次执行，比如由于更换镜像而引起Sandbox重建时。

把 Pod 调度到指定 Node 上

可以通过 nodeSelector、nodeAffinity、podAffinity 以及 Taints 和 tolerations 等来将 Pod 调度到需要的 Node 上。

也可以通过设置 nodeName 参数，将 Pod 调度到指定 node 节点上。

比如，使用 nodeSelector，首先给 Node 加上标签：

```
kubectl label nodes <your-node-name> disktype=ssd
```

接着，指定该 Pod 只想运行在带有 disktype=ssd 标签的 Node 上。

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    env: test
spec:
  containers:
    - name: nginx
      image: nginx
      imagePullPolicy: IfNotPresent
  nodeSelector:
    disktype: ssd
```

nodeSelector

首先给 Node 打上标签：

```
kubectl label nodes node-01 disktype=ssd
```

然后在 daemonset 中指定 nodeSelector 为 disktype=ssd：

```
spec:
```

```
  nodeSelector:
```

```
    disktype: ssd
```

NodeAffinity

NodeAffinity 目前支持两种：requiredDuringSchedulingIgnoredDuringExecution 和 preferredDuringSchedulingIgnoredDuringExecution，分别代表必须满足条件和优选条件。

比如下面的例子代表调度到包含标签 Kubernetes.io/e2e-az-name 并且值为 e2e-az1 或 e2e-az2 的 Node 上，并且优选还带有标签 another-node-label-key=another-node-label-value 的 Node。

```
apiVersion: v1
kind: Pod
metadata:
  name: with-node-affinity
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: kubernetes.io/e2e-az-name
                operator: In
                values:
                  - e2e-az1
                  - e2e-az2
```

```
preferredDuringSchedulingIgnoredDuringExecution:
  - weight: 1
    preference:
      matchExpressions:
        - key: another-node-label-key
          operator: In
          values:
            - another-node-label-value
containers:
  - name: with-node-affinity
    image: gcr.io/google_containers/pause:2.0
```

podAffinity

podAffinity 基于 Pod 的标签来选择 Node，仅调度到满足条件 Pod 所在的 Node 上，支持 podAffinity 和 podAntiAffinity。这个功能比较绕，以下面的例子为例：

如果一个 “Node 所在 Zone 中包含至少一个带有 security=S1 标签且运行中的 Pod”，那么可以调度到该 Node，不调度到 “包含至少一个带有 security=S2 标签且运行中 Pod” 的 Node 上。

podAffinity 示例

```
apiVersion: v1
kind: Pod
metadata:
  name: with-pod-affinity
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: security
                operator: In
                values:
                  - S1
            topologyKey: failure-domain.beta.kubernetes.io/zone
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              podAffinityTerm:
                labelSelector:
                  matchExpressions:
                    - key: security
                      operator: In
                      values:
                        - S2
                  topologyKey: kubernetes.io/hostname
  containers:
    - name: with-pod-affinity
      image: gcr.io/google_containers/pause:2.0
```

Taints 和 Tolerations

Taints 和 Tolerations 用于保证 Pod 不被调度到不合适的 Node 上，其中 Taint 应用于 Node 上，而 Toleration 则应用于 Pod 上。

目前支持的 Taint 类型：

- **NoSchedule**：新的 Pod 不调度到该 Node 上，不影响正在运行的 Pod；
- **PreferNoSchedule**：soft 版的 NoSchedule，尽量不调度到该 Node 上；
- **NoExecute**：新的 Pod 不调度到该 Node 上，并且删除（evict）已在运行的 Pod。Pod 可以增加一个时间（tolerationSeconds）。

然而，当 Pod 的 Tolerations 匹配 Node 的所有 Taints 的时候可以调度到该 Node 上；当 Pod 是已经运行的时候，也不会被删除（evicted）。另外对于 NoExecute，如果 Pod 增加了一个 tolerationSeconds，则会在该时间之后才删除 Pod。

多租户 Kubernetes 集群-计算资源隔离

Kubernetes 集群一般是通用集群，可被所有用户共享，用户无需关心计算节点细节。

但往往某些自带计算资源的客户要求：

- 带着计算资源加入 Kubernetes 集群；
- 要求资源隔离。

实现方案：

- 将要隔离的计算节点打上 Taints；
- 在用户创建 Pod 时，定义 tolerations 来指定要调度到 node taints。

该方案有漏洞吗？如何堵住？

- 其他用户如果可以 get nodes 或者 pods，可以看到 taints 信息，也可以用相同的 tolerations 占用资源。
- 不让用户 get node detail？
- 不让用户 get 别人的 pod detail？
- 企业内部，也可以通过规范管理，通过统计数据看谁占用了哪些 node；
- 数据平面上的隔离还需要其他方案配合。

来自生产系统的经验

- 用户会忘记打 tolerance，导致 pod 无法调度，pending；
 - 新员工常犯的错误，通过聊天机器人的 Q&A 解决；
 - 其他用户会 get node detail，查到 Taints，偷用资源。
-
- 通过 dashboard，能看到哪些用户的什么应用跑在哪些节点上；
 - 对于违规用户，批评教育为主。

优先级调度

从 v1.8 开始，kube-scheduler 支持定义 Pod 的优先级，从而保证高优先级的 Pod 优先调度。开启方法为：

```
apiserver 配置 --feature-gates=PodPriority=true 和 --runtime-  
config=scheduling.k8s.io/v1alpha1=true
```

```
kube-scheduler 配置 --feature-gates=PodPriority=true
```

PriorityClass

在指定 Pod 的优先级之前需要先定义一个 PriorityClass（非 namespace 资源），如：

```
apiVersion: v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
globalDefault: false
description: "This priority class should be used for XYZ service pods only."
```

为 pod 设置 priority

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    env: test
spec:
  containers:
  - name: nginx
    image: nginx
    imagePullPolicy: IfNotPresent
  priorityClassName: high-priority
```

多调度器

如果默认的调度器不满足要求，还可以部署自定义的调度器。并且，在整个集群中还可以同时运行多个调度器实例，通过 `podSpec.schedulerName` 来选择使用哪一个调度器（默认使用内置的调度器）。

来自生产的一些经验

小集群：

100个 node，并发创建 8000 个 Pod 的最大调度耗时大概是 2 分钟左右，发生过 node 删除后，scheduler cache 还有信息的情况，导致 Pod 调度失败。

放大效应：

当一个 node 出现问题所以 load 较小时，通常用户的 Pod 都会优先调度到该 node 上，导致用户所有创建的新 Pod 都失败的情况。

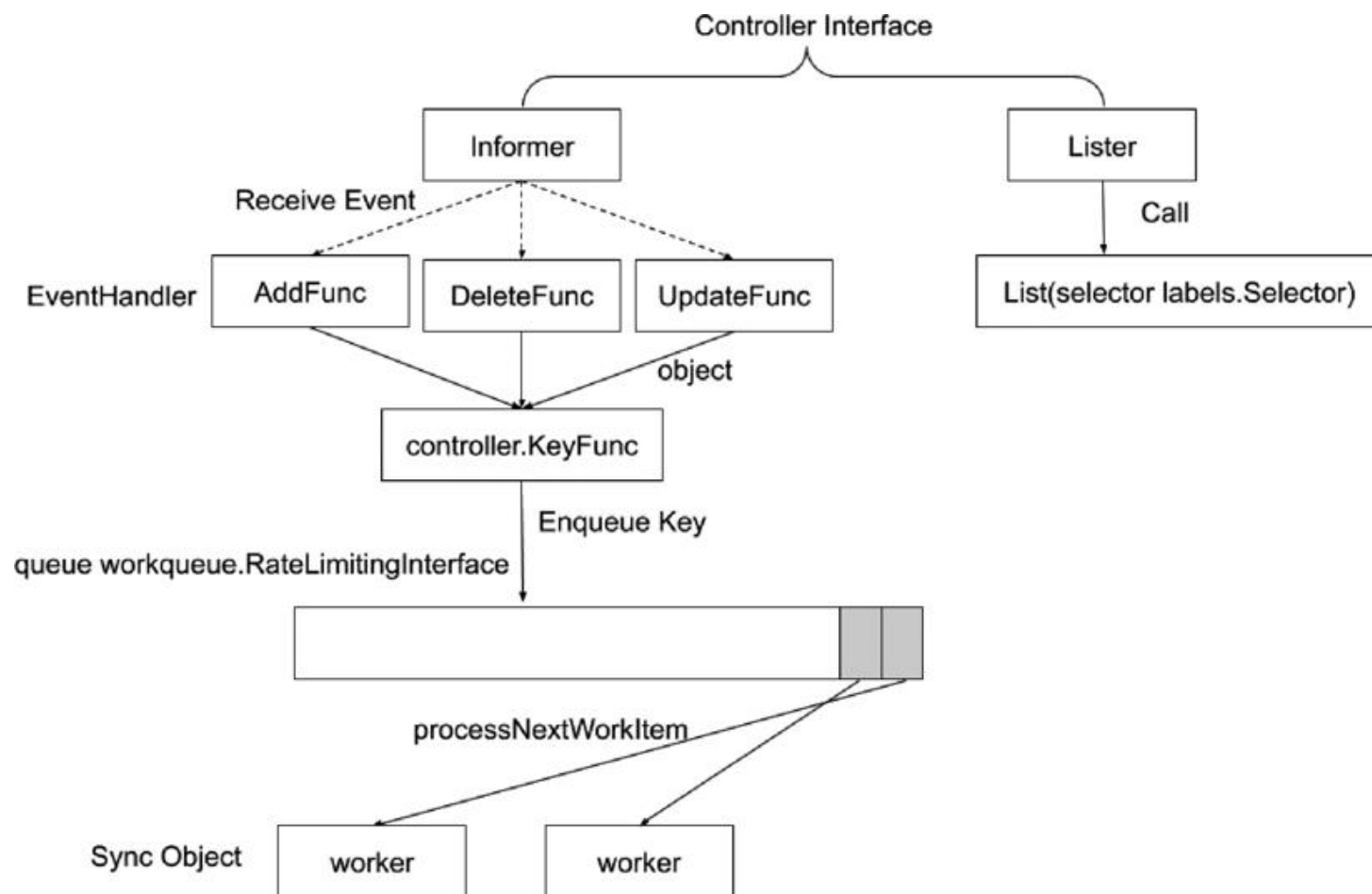
应用炸弹：

存在危险的用户 Pod（比如 fork bomb），在调度到某个 node 上以后，会因为打开文件句柄过多导致 node down 机，Pod 会被 evict 到其他节点，再对其他节点造成伤害，依次循环会导致整个 cluster 所有节点不可用。

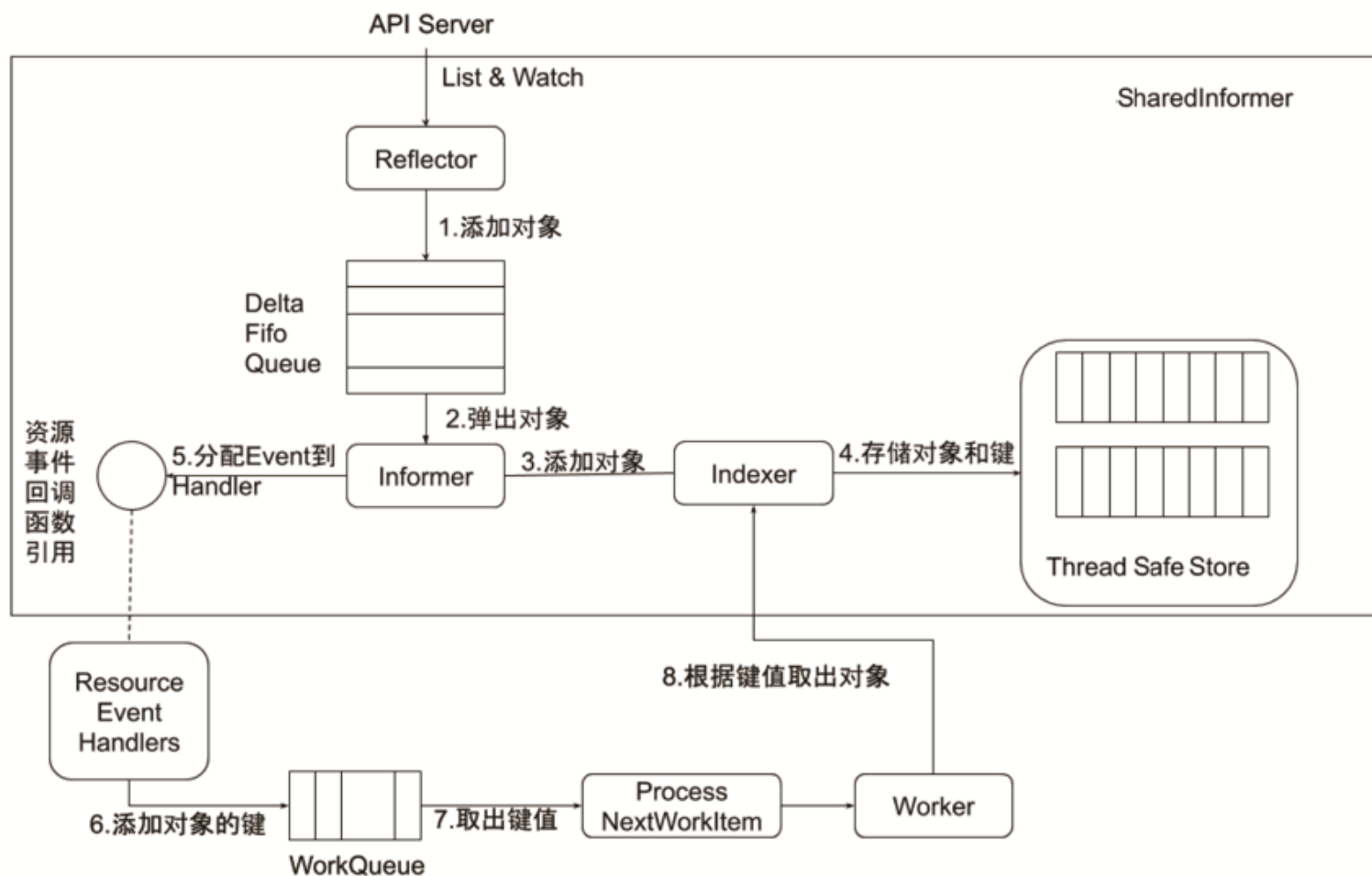
调度器可以说是运营过程中稳定性最好的组件之一，基本没有太大的维护 effort。

2. Controller Manager

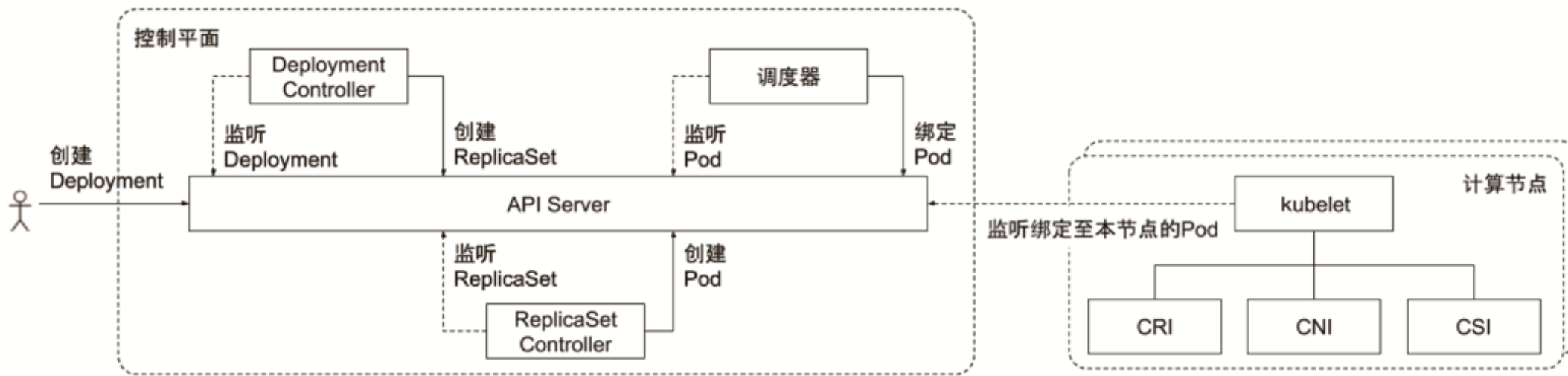
控制器的工作流程



Informer 的内部机制



控制器的协同工作原理



通用 Controller

Job Controller: 处理 job。

Pod AutoScaler: 处理 pod 的自动缩容 / 扩容。

RelicaSet: 依据 Replicaset Spec 创建 Pod。

Service Controller: 为 LoadBalancer type 的 service 创建 LB VIP。

ServiceAccount Controller: 确保 serviceaccount 在当前 namespace 存在。

StatefulSet Controller: 处理 statefulset 中的 pod。

Volume Controller: 依据 PV spec 创建 volume。

Resource quota Controller: 在用户使用资源之后, 更新状态。

通用 Controller

Namespace Controller: 保证namespace删除时, 该namespace下的所有资源都先被删除

Replication Controller: 创建 RC 后, 负责创建 POD。

Node Controller: 维护 node 状态, 处理 evict 请求等。

Daemon Controller: 依据 daemonset 创建 pod。

Deployment Controller: 依据 deployment spec 创建 replicaset。

Endpoint Controller: 依据 service spec 创建 endpoint, 依据 podip 更新 endpoint。

Garbage Collector: 处理级联删除, 比如删除 deployment 的同时删除 replicaset 以及 pod。

CronJob Controller: 处理 cronjob。

Cloud Controller Manager

什么时候需要 cloud controller manager?

Cloud Controller Manager 自 Kubernetes1.6 开始，从 kube-controller-manager 中分离出来，主要因为 Cloud Controller Manager 往往需要跟企业 cloud 做深度集成，release cycle 跟Kubernetes 相对独立。

与 Kubernetes 核心管理组件一起升级是一件费时费力的事。

通常 cloud controller manager 需要：

- 认证授权：企业 cloud 往往需要认证信息，Kubernetes 要与 Cloud API 通信，需要获取 cloud系统里的 ServiceAccount；
- Cloud controller manager 本身作为一个用户态的 component，需要在 Kubernetes 中有正确的 RBAC 设置，获得资源操作权限；
- 高可用：需要通过 leader election 来确保 cloud controller manger 高可用。

Cloud controller manager 的配置



cloud controller manager 是从老版本的 APIServer 分离出来的。

Kube-APIServer 和 kube-controller-manager 中一定不能指定 cloud-provider, 否则会加载内置的 cloud controller manager。

Kubelet 要配置 --cloud-provider=external。

Cloud Controller Manager 主要支持:

Node controller: 访问 cloud API, 来更新 node 状态; 在 cloud 删除该节点以后, 从 kubernetes 删除 node;

Service controller: 负责配置为 loadbalancer 类型的服务配置 LB VIP;

Route Controller: 在 cloud 环境配置路由;

可以自定义任何需要的 Cloud Controller。

需要定制的 Cloud controller

- Ingress controller;
- Service Controller;
- 自主研发的 controller, 比如之前提到的:
 - RBAC controller;
 - Account controller。

来自生产的经验

保护好 controller manager 的 kubeconfig:

此 kubeconfig 拥有所有资源的所有操作权限，防止普通用户通过 `kubectl exec kube-controller-manager cat` 获取该文件。

用户可能做任何你想象不到的操作，然后来找你 support。

Pod evict 后 IP 发生变化，但 endpoint 中的 address 更新失败：

分析 stacktrace 发现 endpoint 在更新 LoadBalancer 时调用 gophercloud 连接 hang 住，导致 endpoint worker 线程全部卡死。

确保 scheduler 和 controller 的高可用



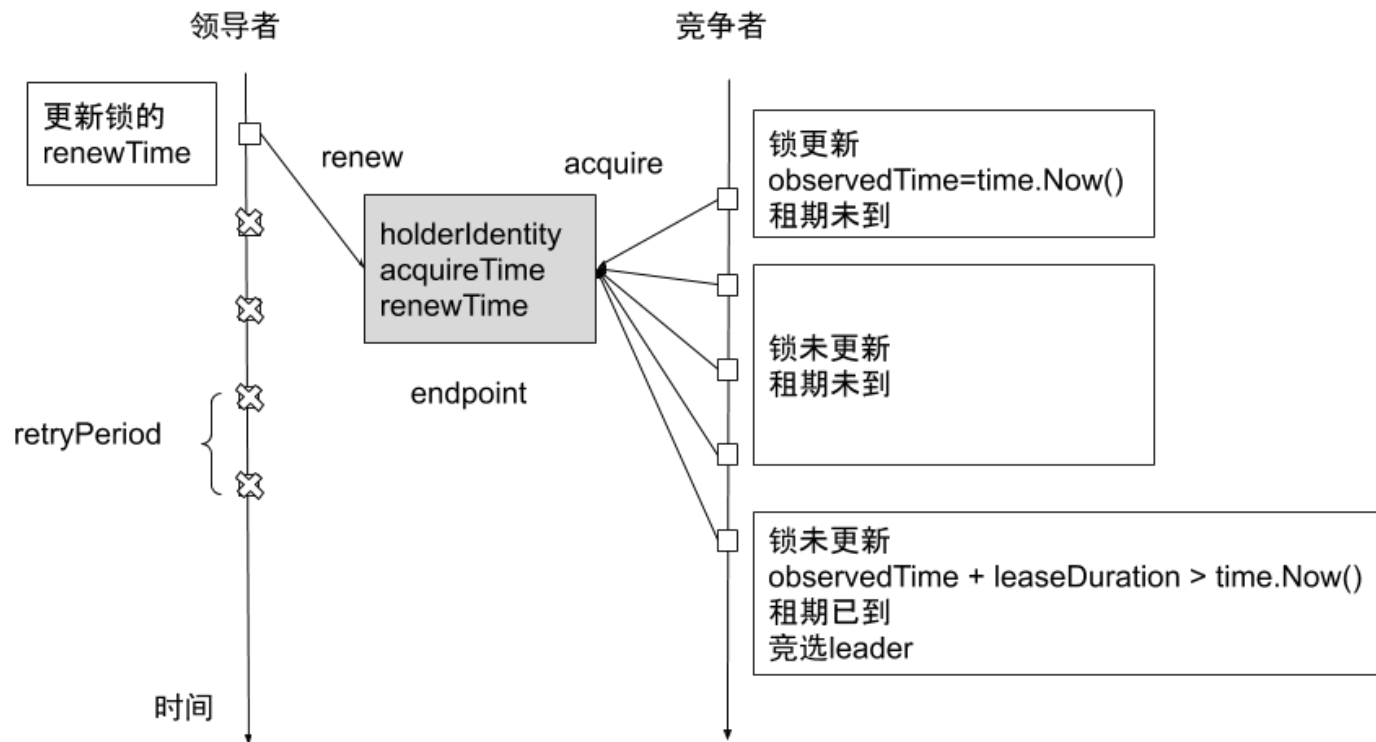
Leader Election

Kubernetes 提供基于 configmap 和 endpoint 的 leader election 类库

Kubernetes 采用 leader election 模式启动 component 后，会创建对应 endpoint，并把当前的 leader 信息 annotate 到 endpoint 上

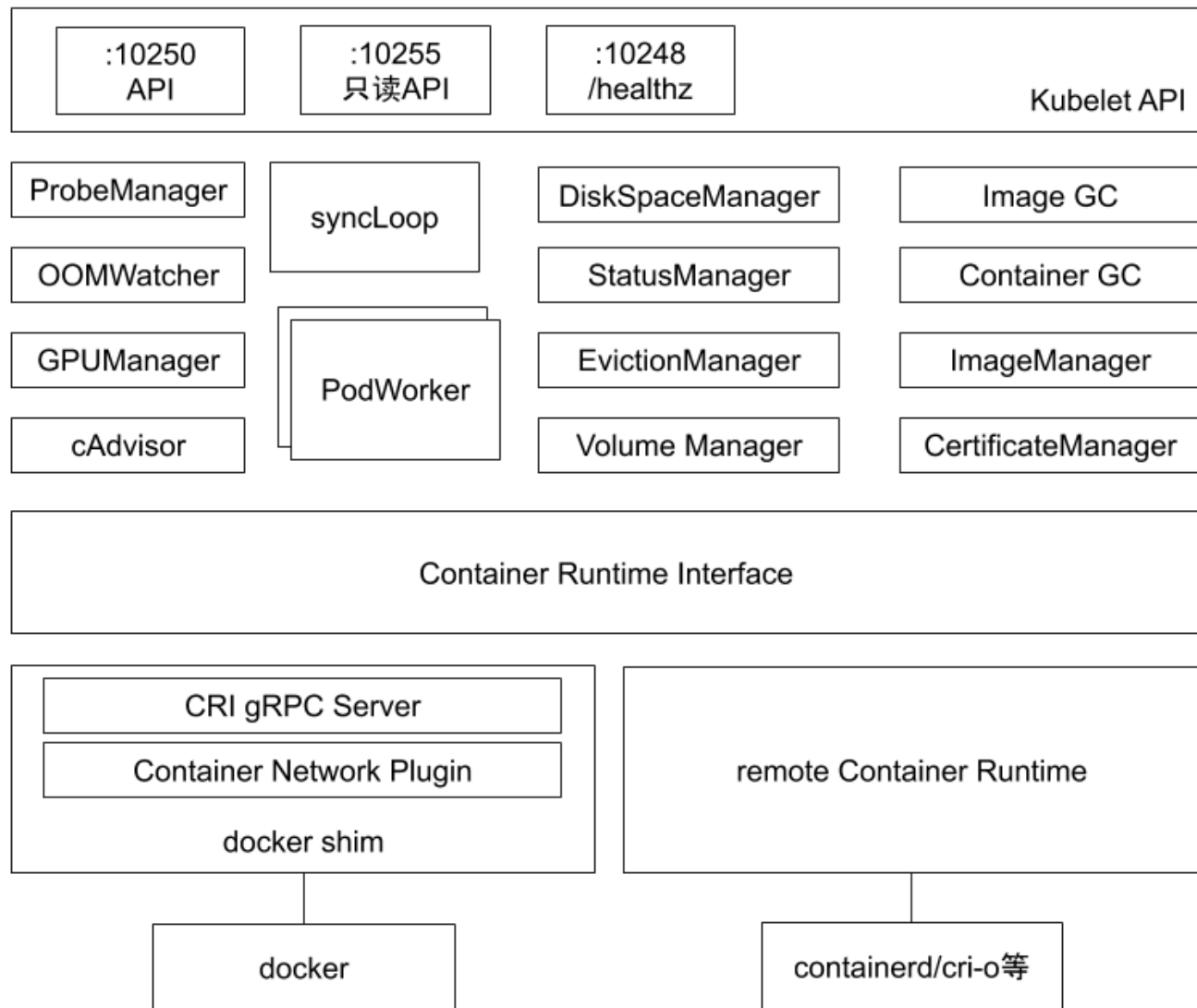
```
apiVersion: v1
kind: Endpoints
metadata:
  annotations:
    control-plane.alpha.kubernetes.io/leader:
      '{"holderIdentity":"minikube","leaseDurationSeconds":15,"acquireTime":"2018-04-05T17:31:29Z","renewTime":"2018-04-07T07:18:39Z","leaderTransitions":0}'
  creationTimestamp: 2018-04-05T17:31:29Z
  name: kube-scheduler
  namespace: kube-system
  resourceVersion: "138930"
  selfLink: /api/v1/namespaces/kube-system/endpoints/kube-scheduler
  uid: 2d12578d-38f7-11e8-8df0-0800275259e5
subsets: null
```

Leader Election

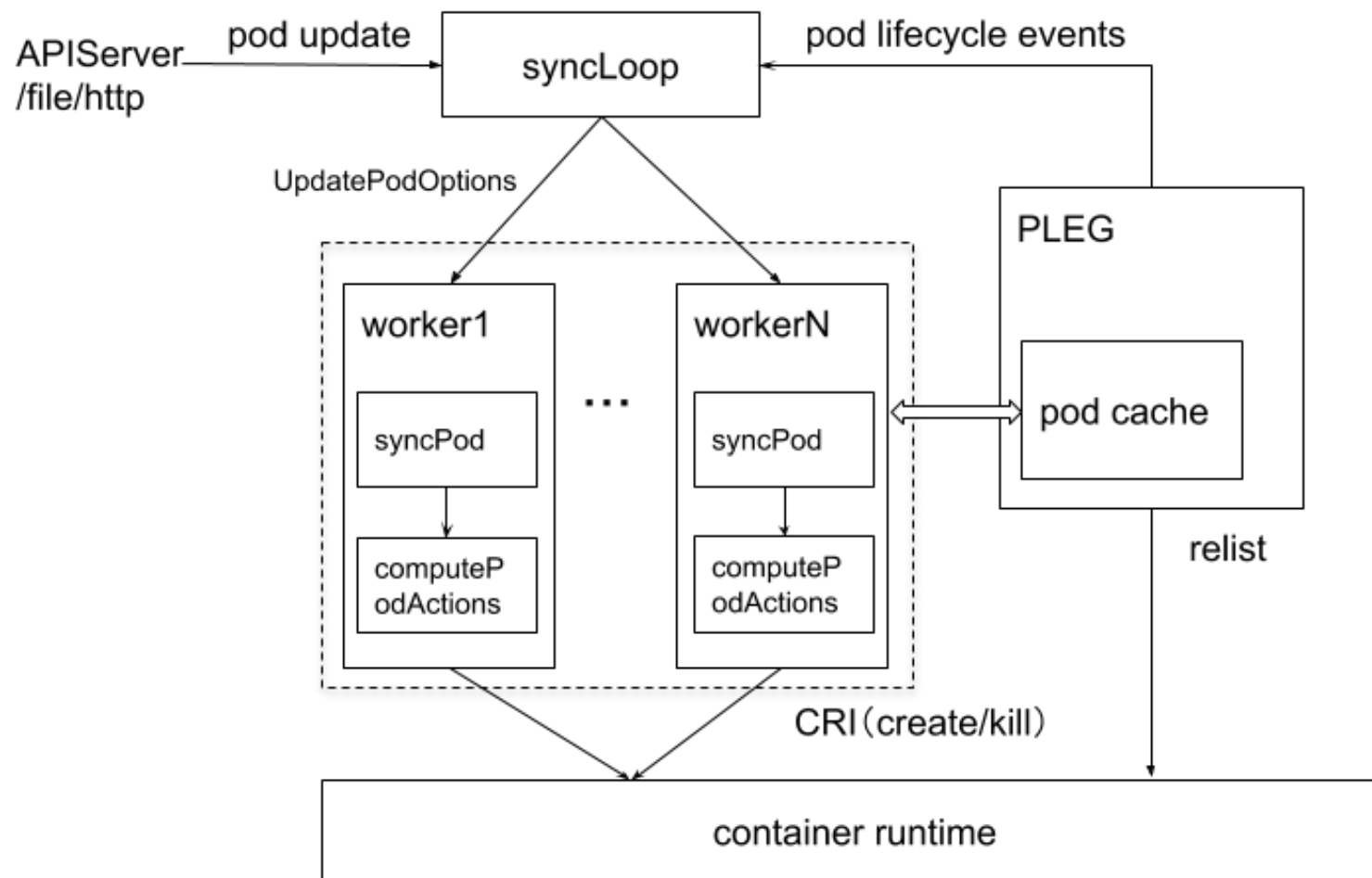


3. kubelet

kubelet 架构



kubelet 管理 Pod 的核心流程



kubelet

每个节点上都运行一个 kubelet 服务进程，默认监听 10250 端口。

- 接收并执行 master 发来的指令；
- 管理 Pod 及 Pod 中的容器；
- 每个 kubelet 进程会在 API Server 上注册节点自身信息，定期向 master 节点汇报节点的资源使用情况，并通过 cAdvisor 监控节点和容器的资源。

节点管理主要是节点自注册和节点状态更新：

Kubelet 可以通过设置启动参数 `--register-node` 来确定是否向 API Server 注册自己；

如果 Kubelet 没有选择自注册模式，则需要用户自己配置 Node 资源信息，同时需要告知 Kubelet 集群上的 API Server 的位置；

Kubelet 在启动时通过 API Server 注册节点信息，并定时向 API Server 发送节点新消息，API Server 在接收到新消息后，将信息写入 etcd。

Pod 管理

获取 Pod 清单：

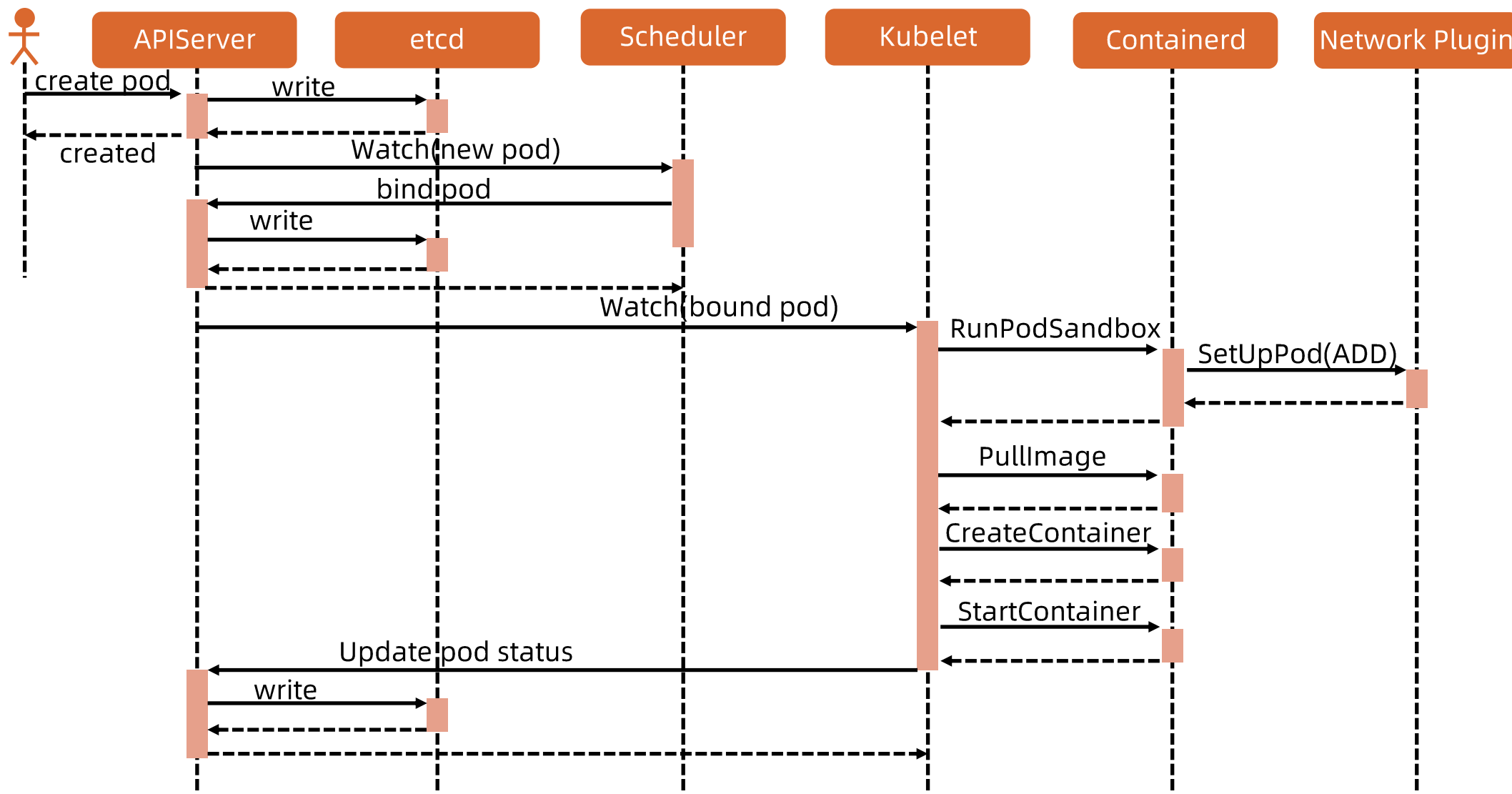
文件：启动参数 `--config` 指定的配置目录下的文件(默认 `/etc/Kubernetes/manifests/`)。该文件每20秒重新检查一次（可配置）。

HTTP endpoint（URL）：启动参数 `--manifest-url` 设置。每 20 秒检查一次这个端点（可配置）。

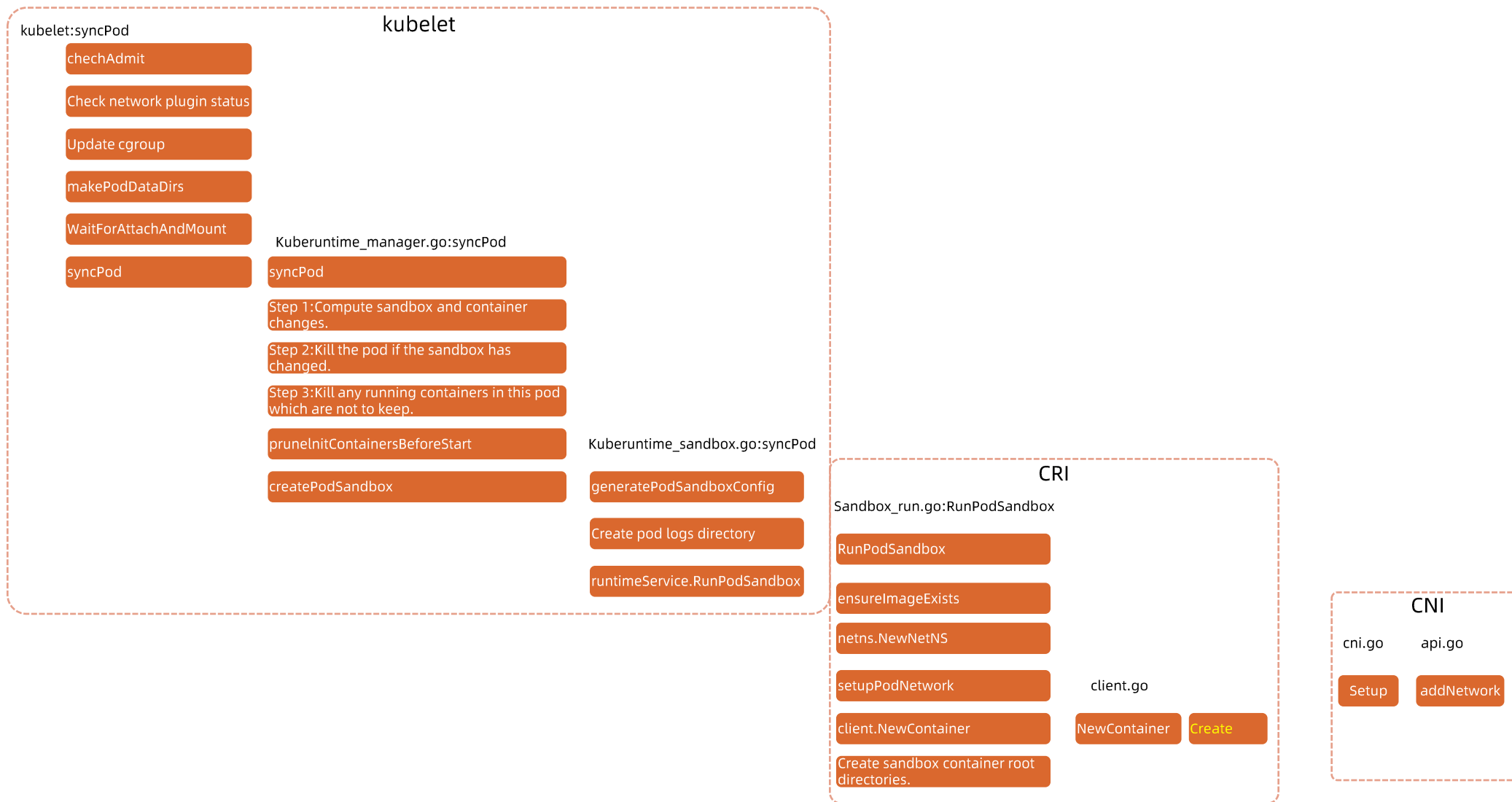
API Server：通过 API Server 监听 etcd 目录，同步 Pod 清单。

HTTP server：kubelet 侦听 HTTP 请求，并响应简单的 API 以提交新的 Pod 清单。

Pod启动流程



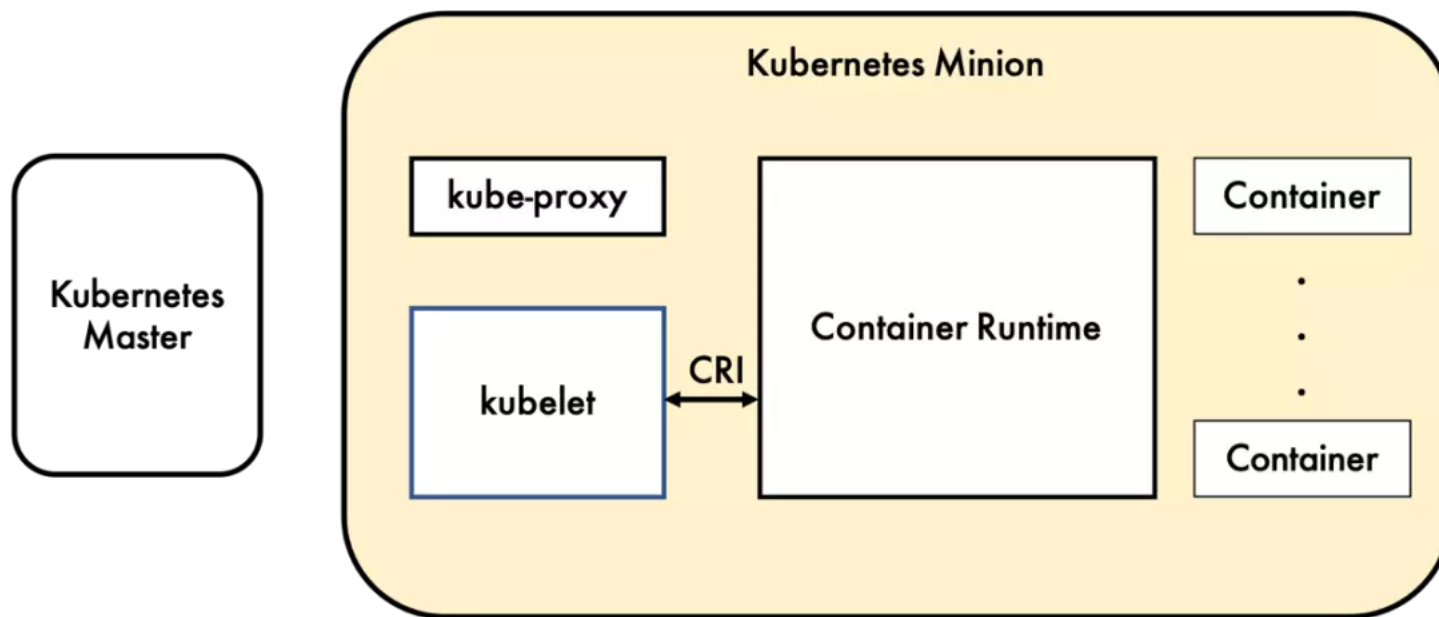
Kubelet启动Pod的流程



4. CRI

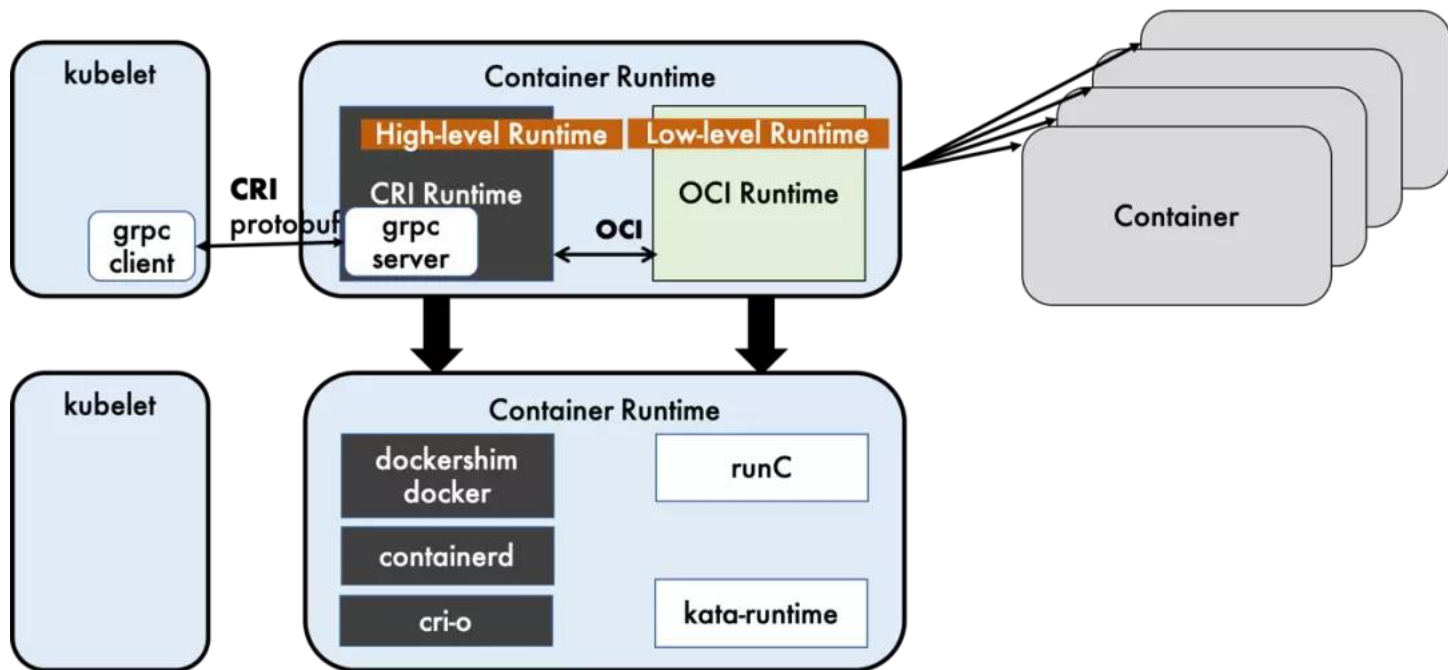
CRI

容器运行时（Container Runtime），运行于 Kubernetes（k8s）集群的每个节点中，负责容器的整个生命周期。其中 Docker 是目前应用最广的。随着容器云的发展，越来越多的容器运行时涌现。为了解决这些容器运行时和 Kubernetes 的集成问题，在 Kubernetes 1.5 版本中，社区推出了 CRI（Container Runtime Interface，容器运行时接口）以支持更多的容器运行时。



CRI

CRI 是 Kubernetes 定义的一组 gRPC 服务。kubelet 作为客户端，基于 gRPC 框架，通过 Socket 和容器运行时通信。它包括两类服务：镜像服务（Image Service）和运行时服务（Runtime Service）。镜像服务提供下载、检查和删除镜像的远程程序调用。运行时服务包含用于管理容器生命周期，以及与容器交互的调用（exec / attach / port-forward）的远程程序调用。



运行时的层级

Dockershim, containerd 和 CRI-O 都是遵循 CRI 的容器运行时，我们称他们为 **高层级运行时 (High-level Runtime)**。

OCI (Open Container Initiative, 开放容器计划) 定义了创建容器的格式和运行时的开源行业标准，包括 镜像规范 (Image Specification) 和运行时规范(Runtime Specification)。

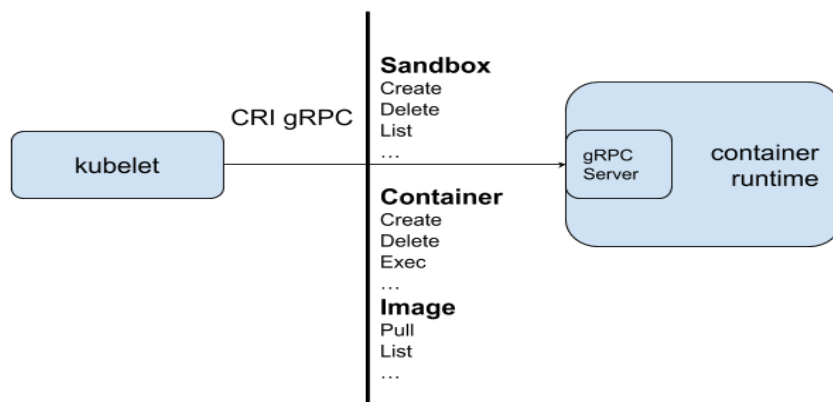
镜像规范定义了 OCI 镜像的标准。高层级运行时将会下载一个 OCI 镜像，并把它解压成 OCI 运行时文件系统包 (filesystem bundle)。

运行时规范则描述了如何从 OCI 运行时文件系统包运行容器程序，并且定义它的配置、运行环境和生命周期。如何为新容器设置 命名空间 (namespaces) 和 控制组 (cgroups)，以及挂载根文件系统等等操作，都是在这里定义的。它的一个参考实现是 runC。我们称其为 **低层级运行时 (Low-level Runtime)**。除 runC 以外，也有很多其他的运行时遵循 OCI 标准，例如 kata-runtime。

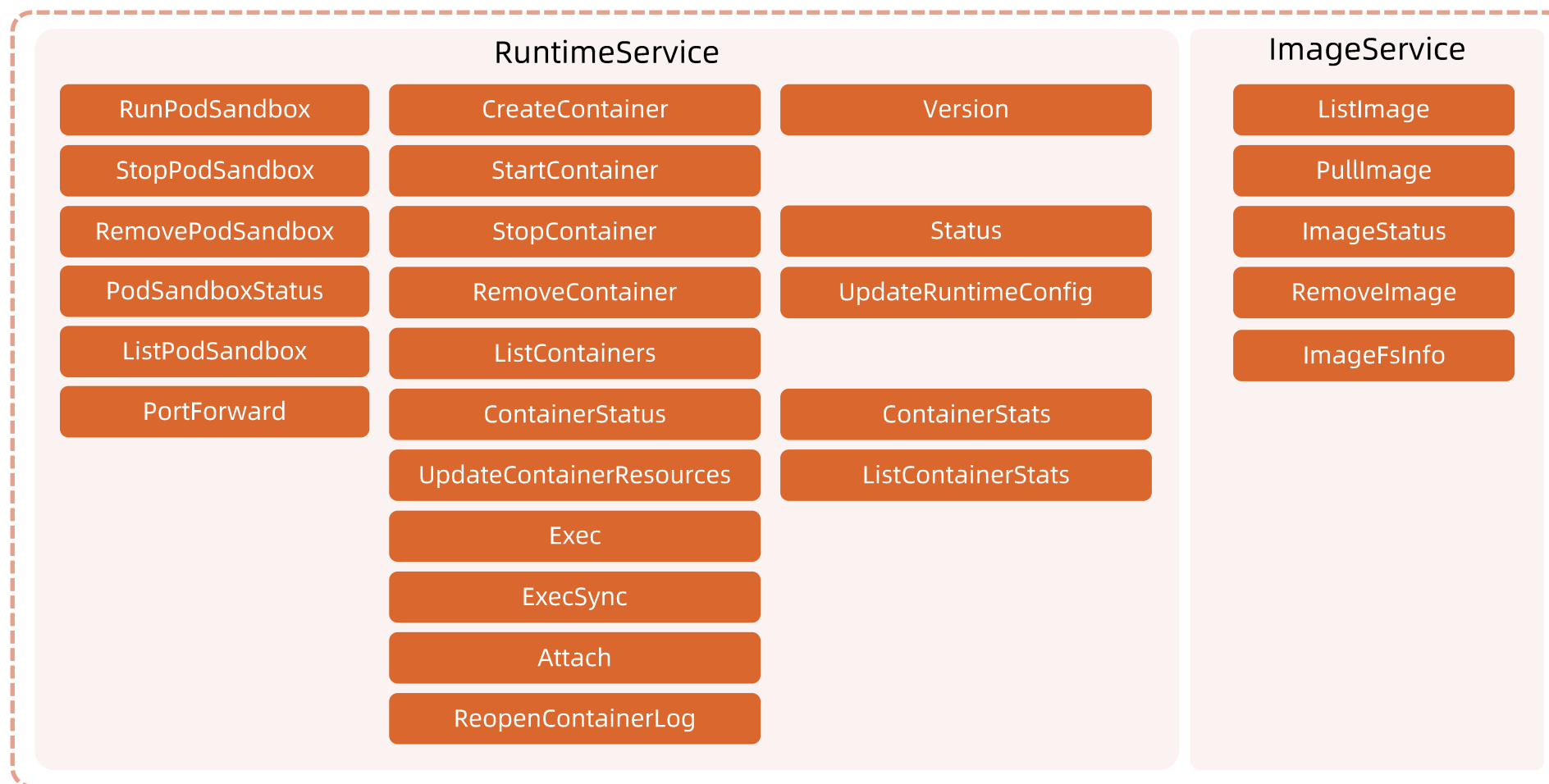
容器运行时是真正起删和管理容器的组件。容器运行时可以分为高层和低层的运行时。高层运行时主要包括 Docker, containerd 和 CRI-O, 低层的运行时, 包含了 runc, kata, 以及 gVisor。低层运行时 kata 和 gVisor 都还处于小规模落地或者实验阶段, 其生态成熟度和使用案例都比较欠缺, 所以除非有特殊的需求, 否则 runc 几乎是必然的选择。因此在对容器运行时的选择上, 主要是聚焦于上层运行时的选择。

Docker 内部关于容器运行时功能的核心组件是 containerd, 后来 containerd 也可直接和 kubelet 通过 CRI 对接, 独立在 Kubernetes 中使用。相对于 Docker 而言, containerd 减少了 Docker 所需的处理模块 Dockerd 和 Docker-shim, 并且对 Docker 支持的存储驱动进行了优化, 因此在容器的创建启动停止和删除, 以及对镜像的拉取上, 都具有性能上的优势。架构的简化同时也带来了维护的便利。当然 Docker 也具有很多 containerd 不具有的功能, 例如支持 zfs 存储驱动, 支持对日志的大小和文件限制, 在以 overlayfs2 做存储驱动的情况下, 可以通过 xfs_quota 来对容器的可写层进行大小限制等。尽管如此, containerd 目前也基本上能够满足容器的众多管理需求, 所以将它作为运行时的也越来越多。

Kubelet 和运行时的关系:



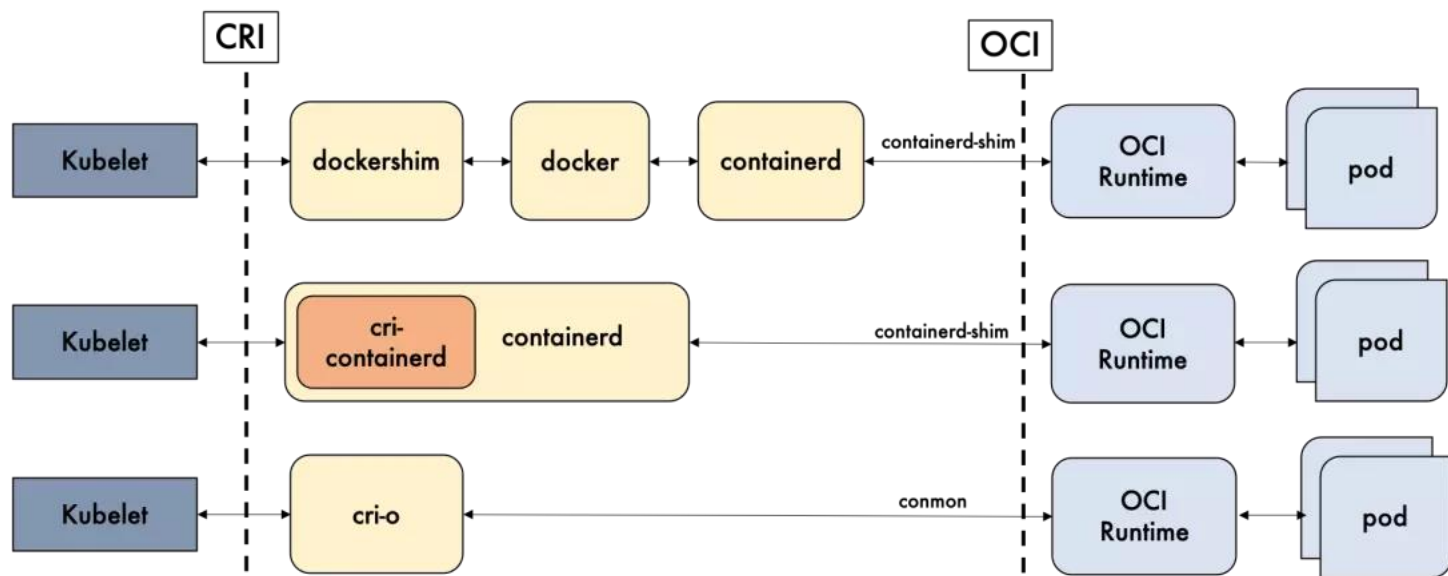
CRI



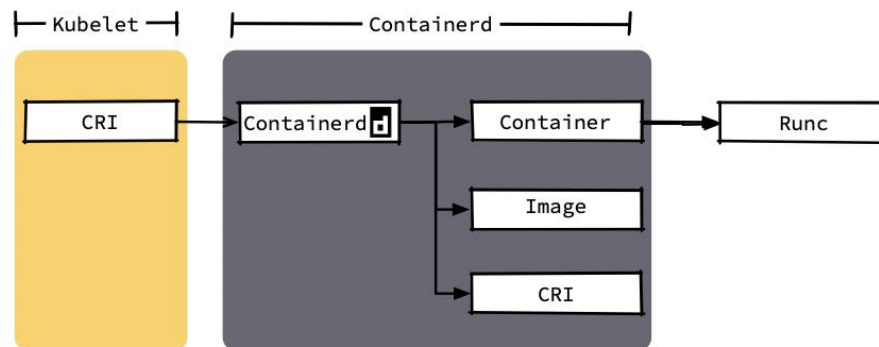
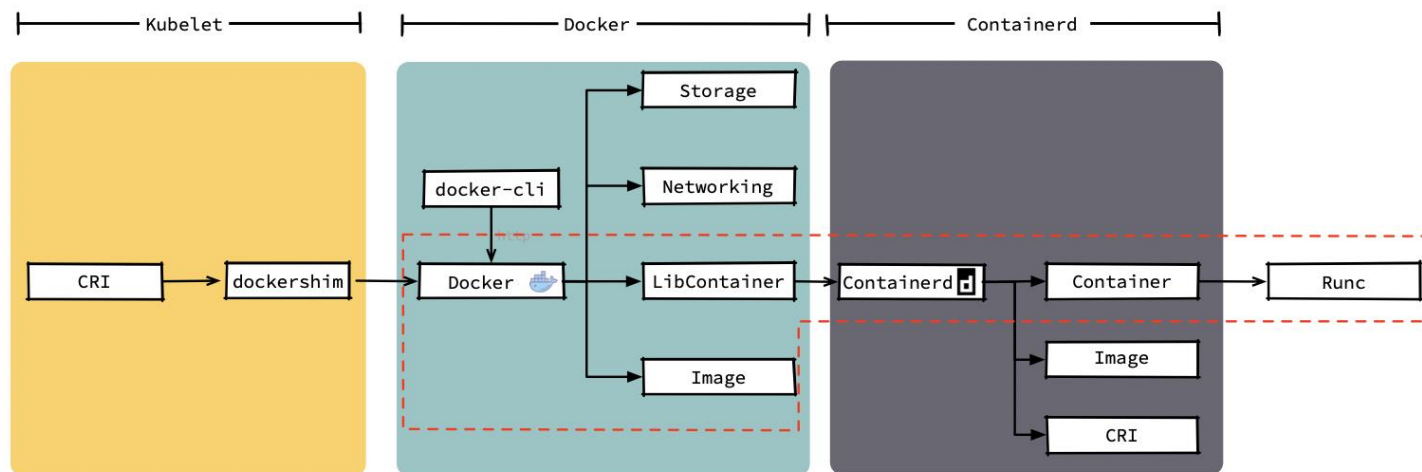
开源运行时的比较

Docker 的多层封装和调用，导致其在可维护性上略逊一筹，增加了线上问题的定位难度；几乎除了重启 Docker，我们就毫无他法了。

containerd 和 CRI-O 的方案比起 Docker 简洁很多。



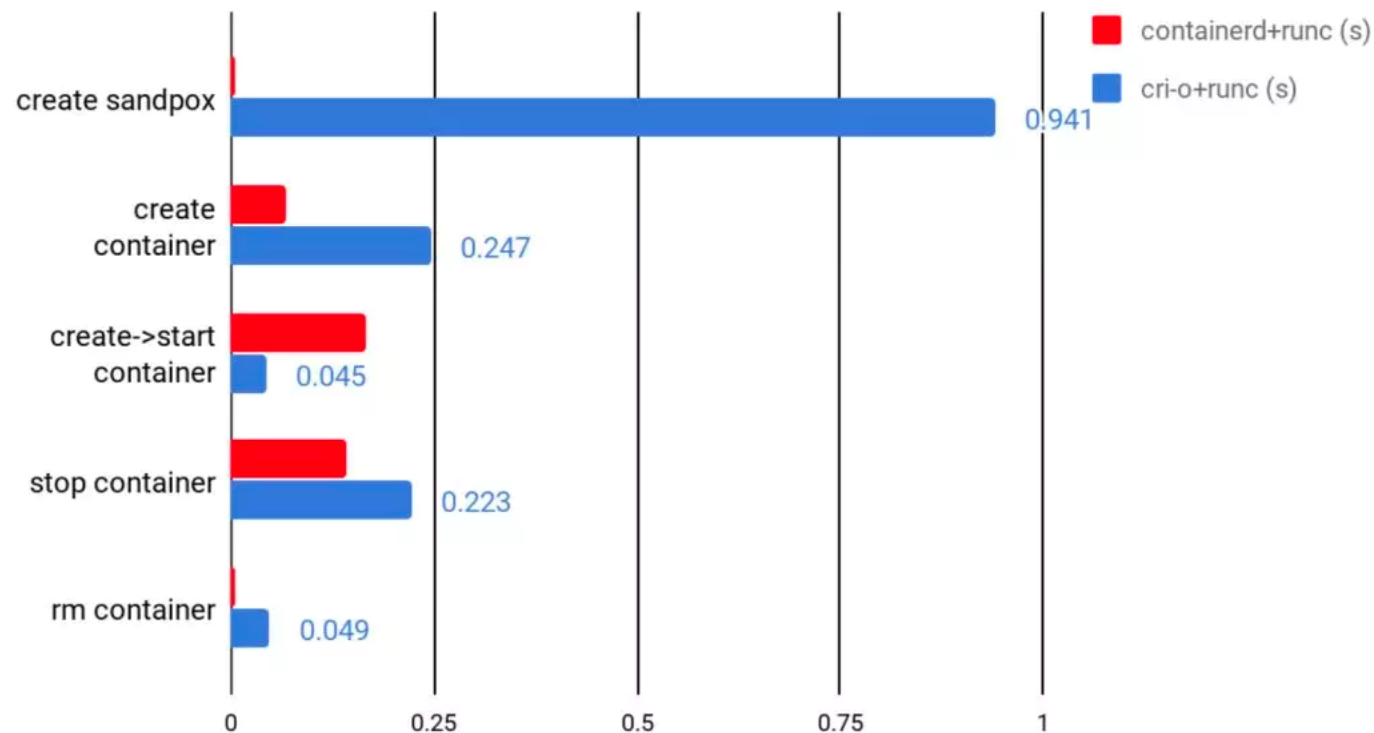
Docker和containerd的差异细节



多种运行时性能比较

containerd 在各个方面都表现良好，除了启动容器这项。从总用时来看，containerd 的用时还是要比 CRI-O 要短的

runC Performance



运行时优劣对比

功能性来讲，containerd 和 CRI-O 都符合 CRI 和 OCI 的标准；
在稳定性上，containerd 略胜一筹；
从性能上讲，containerd 胜出。

	containerd	CRI-O	备注
性能	更优	优	
功能	优	优	CRI与OCI兼容
稳定性	稳定	未知	

5. CNI

Kubernetes 网络模型设计的基础原则是：

所有的 Pod 能够不通过 NAT 就能相互访问。

所有的节点能够不通过 NAT 就能相互访问。

容器内看见的 IP 地址和外部组件看到的容器 IP 是一样的。

Kubernetes 的集群里，IP 地址是以 Pod 为单位进行分配的，每个 Pod 都拥有一个独立的 IP 地址。一个 Pod 内部的所有容器共享一个网络栈，即宿主机上的一个网络命名空间，包括它们的 IP 地址、网络设备、配置等都是共享的。也就是说，Pod 里面的所有容器能通过 localhost:port 来连接对方。在 Kubernetes 中，提供了一个轻量的通用容器网络接口 CNI（Container Network Interface），专门用于设置和删除容器的网络连通性。容器运行时通过 CNI 调用网络插件来完成容器的网络设置。

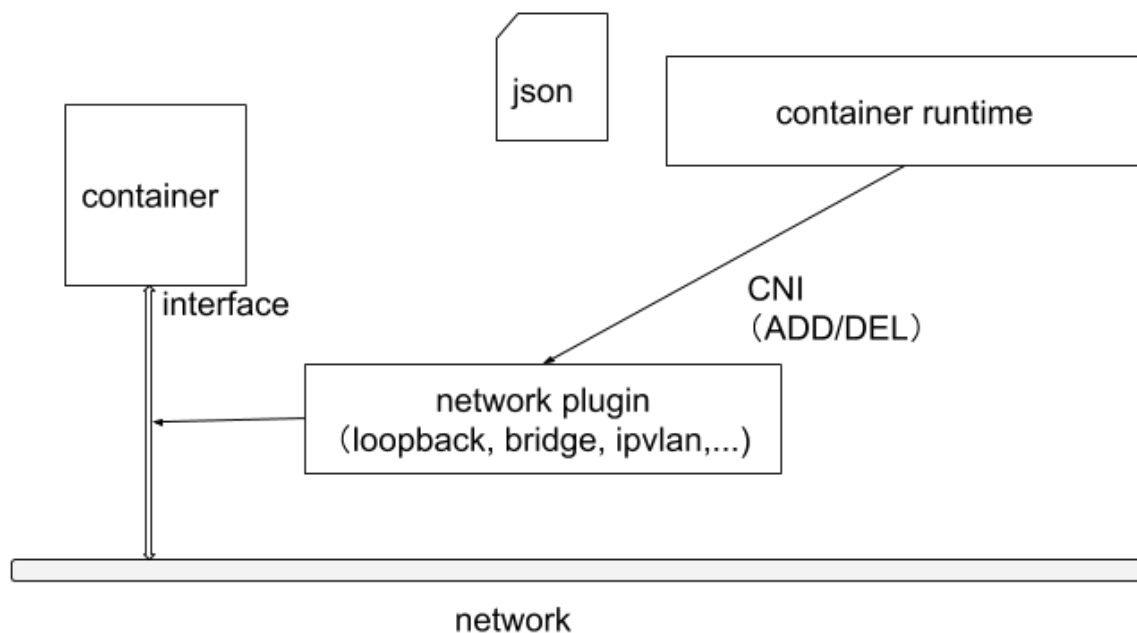
CNI插件分类和常见插件

- IPAM: IP 地址分配
- 主插件: 网卡设置
 - bridge: 创建一个网桥, 并把主机端口和容器端口插入网桥
 - ipvlan: 为容器添加ipvlan网口
 - loopback: 设置loopback网口
- Meta: 附加功能
 - portmap: 设置主机端口和容器端口映射
 - bandwidth: 利用Linux Traffic Control限流
 - firewall: 通过iptables或firewalld为容器设置防火墙规则

<https://github.com/containernetworking/plugins>

CNI 插件运行机制

容器运行时在启动时会从 CNI 的配置目录中读取 JSON 格式的配置文件，文件后缀为 “.conf” “.conflist” “.json” 。如果配置目录中包含多个文件，一般情况下，会以名字排序选用第一个配置文件作为默认的网络配置，并加载获取其中指定的 CNI 插件名称和配置参数。



CNI 的运行机制

关于容器网络管理，容器运行时一般需要配置两个参数 `--cni-bin-dir` 和 `--cni-conf-dir`。有一种特殊情况，kubelet 内置的 Docker 作为容器运行时，是由 kubelet 来查找 CNI 插件的，运行插件来为容器设置网络，这两个参数应该配置在 kubelet 处：

`cni-bin-dir`：网络插件的可执行文件所在目录。默认是 `/opt/cni/bin`。

`cni-conf-dir`：网络插件的配置文件所在目录。默认是 `/etc/cni/net.d`。

CNI 插件设计考量

容器运行时必须在调用任何插件之前为容器创建一个新的网络命名空间。

容器运行时必须决定这个容器属于哪些网络，针对每个网络，哪些插件必须要执行。

容器运行时必须加载配置文件，并确定设置网络时哪些插件必须被执行。

网络配置采用 JSON 格式，可以很容易地存储在文件中。

容器运行时必须按顺序执行配置文件里相应的插件。

CNI 插件设计考量

在完成容器生命周期后，容器运行时必须按照与执行添加容器相反的顺序执行插件，以便将容器与网络断开连接。

容器运行时被同一容器调用时不能并行操作，但被不同的容器调用时，允许并行操作。

容器运行时针对一个容器必须按顺序执行 ADD 和 DEL 操作，ADD 后面总是跟着相应的 DEL。DEL 可能跟着额外的 DEL，插件应该允许处理多个 DEL。

容器必须由 ContainerID 来唯一标识，需要存储状态的插件需要使用网络名称、容器 ID 和网络接口组成的主 key 用于索引。

容器运行时针对同一个网络、同一个容器、同一个网络接口，不能连续调用两次 ADD 命令。

打通主机层网络

CNI 插件外，Kubernetes 还需要标准的 CNI 插件 `io`，最低版本为 0.2.0 版本。网络插件除支持设置和清理 Pod 网络接口外，该插件还需要支持 `Iptables`。如果 `Kube-proxy` 工作在 `Iptables` 模式，网络插件需要确保容器流量能使用 `Iptables` 转发。例如，如果网络插件将容器连接到 Linux 网桥，必须将 `net/bridge/bridge-nf-call-iptables` 参数 `sysctl` 设置为 1，网桥上数据包将遍历 `Iptables` 规则。如果插件不使用 Linux 桥接器（而是类似 `Open vSwitch` 或其他某种机制的插件），则应确保容器流量被正确设置了路由。

CNI Plugin

ContainerNetworking 组维护了一些 CNI 插件，包括网络接口创建的 bridge、ipvlan、loopback、macvlan、ptp、host-device 等，IP 地址分配的 DHCP、host-local 和 static，其他的 Flannel、tunning、portmap、firewall 等。

社区还有些第三方网络策略方面的插件，例如 Calico、Cilium 和 Weave 等。可用选项的多样性意味着大多数用户将能够找到适合其当前需求和部署环境的 CNI 插件，并在情况变化时迅捷转换解决方案。

Flannel

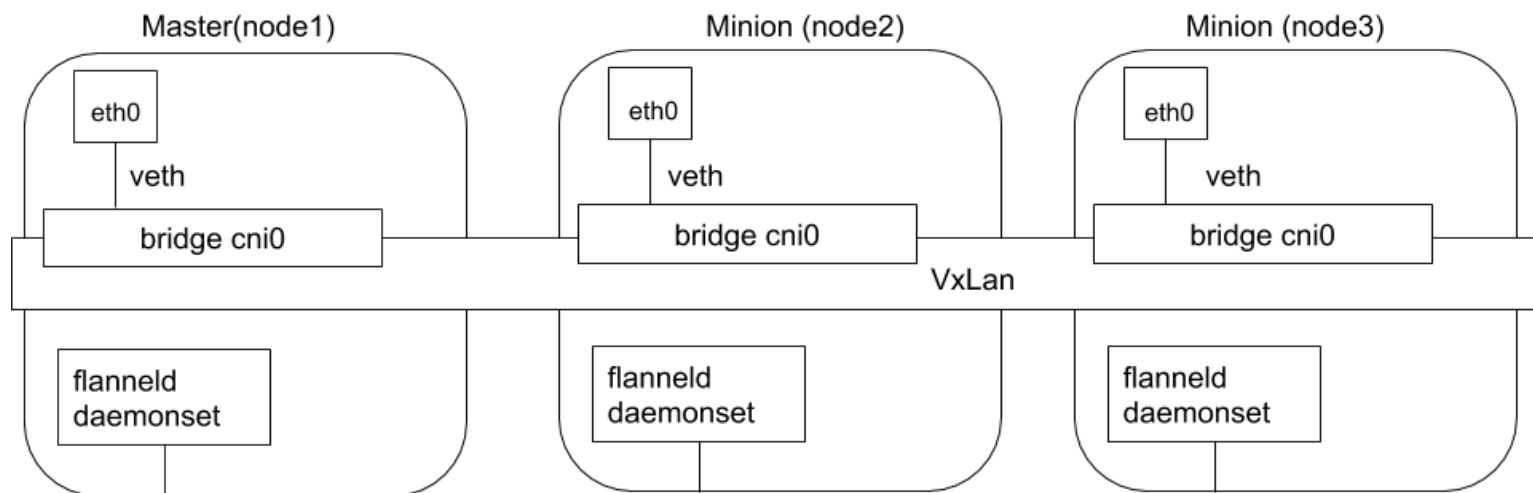
Flannel 是由 CoreOS 开发的项目，是 CNI 插件早期的入门产品，简单易用。

Flannel 使用 Kubernetes 集群的现有 etcd 集群来存储其状态信息，从而不必提供专用的数据存储，只需要在每个节点上运行 flanneld 来守护进程。

每个节点都被分配一个子网，为该节点上的 Pod 分配 IP 地址。

同一主机内的 Pod 可以使用网桥进行通信，而不同主机上的 Pod 将通过 flanneld 将其流量封装在 UDP 数据包中，以路由到适当的目的地。

封装方式默认和推荐的方法是使用 VXLAN，因为它具有良好的性能，并且比其他选项要少些人为干预。虽然使用 VXLAN 之类的技术封装的解决方案效果很好，但缺点就是该过程使流量跟踪变得困难。



Calico 以其性能、灵活性和网络策略而闻名，不仅涉及在主机和 Pod 之间提供网络连接，而且还涉及网络安全性和策略管理。

对于同网段通信，基于第 3 层，Calico 使用 BGP 路由协议在主机之间路由数据包，使用 BGP 路由协议也意味着数据包在主机之间移动时不需要包装在额外的封装层中。

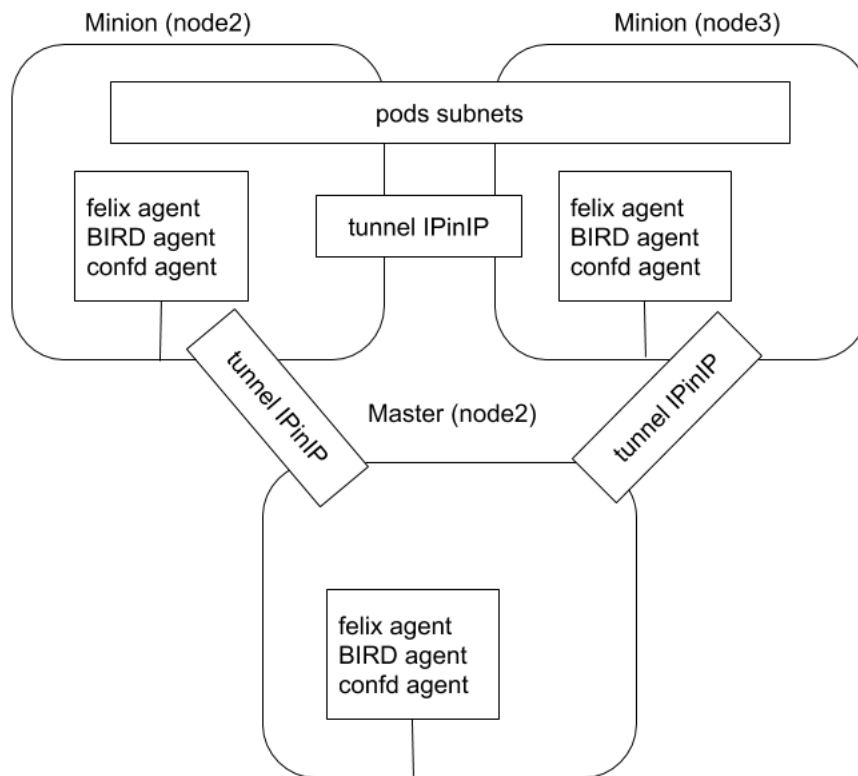
对于跨网段通信，基于 IPinIP 使用虚拟网卡设备 tunl0，用一个 IP 数据包封装另一个 IP 数据包，外层 IP 数据包头的源地址为隧道入口设备的 IP 地址，目标地址为隧道出口设备的 IP 地址。

网络策略是 Calico 最受欢迎的功能之一，使用 ACLs 协议和 kube-proxy 来创建 iptables 过滤规则，从而实现隔离容器网络的目的。

此外，Calico 还可以与服务网格 Istio 集成，在服务网格层和网络基础结构层上解释和实施集群中工作负载的策略。这意味着您可以配置功能强大的规则，以描述 Pod 应该如何发送和接收流量，提高安全性及加强对网络环境的控制。

Calico 属于完全分布式的横向扩展结构，允许开发人员和管理员快速和平稳地扩展部署规模。对于性能和功能（如网络策略）要求高的环境，Calico 是一个不错选择。

Calico组件



Calico初始化

配置和CNI二进制文件由initContainer推送

- command:

- /opt/cni/bin/install

- env:

- name: CNI_CONF_NAME

- value: 10-calico.conflist

- name: SLEEP

- value: "false"

- name: CNI_NET_DIR

- value: /etc/cni/net.d

- name: CNI_NETWORK_CONFIG

- valueFrom:

- configMapKeyRef:

- key: config

- name: cni-config

- name:
KUBERNETES_SERVICE_HOST

- value: 10.96.0.1

- name:
KUBERNETES_SERVICE_PORT

- value: "443"

- image: docker.io/calico/cni:v3.20.1

- imagePullPolicy: IfNotPresent

- name: install-cni

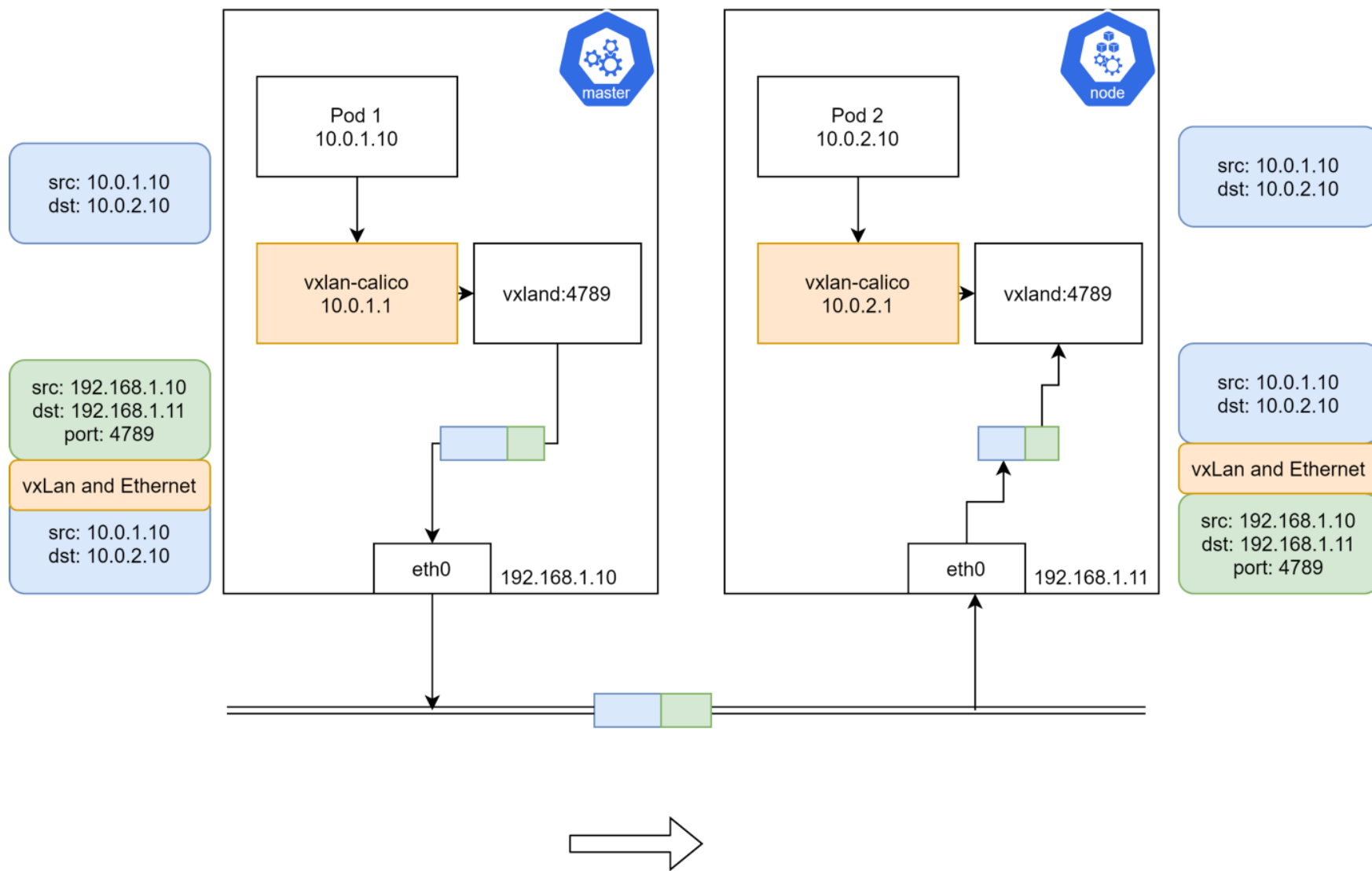
Calico配置一览

```
{
  "name": "k8s-pod-network",
  "cniVersion": "0.3.1",
  "plugins": [
    {
      "type": "calico",
      "datastore_type": "kubernetes",
      "mtu": 0,
      "nodename_file_optional": false,
      "log_level": "Info",
      "log_file_path": "/var/log/calico/cni/cni.log",
      "ipam": { "type": "calico-ipam", "assign_ipv4": "true",
"assign_ipv6": "false"},
      "container_settings": {
        "allow_ip_forwarding": false
      },
```

```
"policy": {
  "type": "k8s"
},
"kubernetes": {
  "k8s_api_root": "https://10.96.0.1:443",
  "kubeconfig": "/etc/cni/net.d/calico-kubeconfig"
},
{
  "type": "bandwidth",
  "capabilities": {"bandwidth": true}
},
{"type": "portmap", "snat": true, "capabilities": {"portMappings":
true}}
]
}
```

Calico VXLAN

VxLan - Calico



IPPool

IPPool用来定义一个集群的预定义IP段

apiVersion: crd.projectcalico.org/v1

kind: IPPool

metadata:

name: default-ipv4-ippool

spec:

blockSize: 26

cidr: 192.168.0.0/16

ipipMode: Never

natOutgoing: true

nodeSelector: all()

vxlanMode: CrossSubnet

IPAMBlock

IPAMBlock 用来定义每个主机预分配的IP段

apiVersion: crd.projectcalico.org/v1

kind: IPAMBlock

metadata:

annotations:

name: 192-168-119-64-26

spec:

affinity: host:cadmin

allocations:

- null
- 0
- null
- 1
- 2
- 3

attributes:

- handle_id: vxlan-tunnel-addr-cadmin

secondary:

node: cadmin

type: vxlanTunnelAddress

- handle_id: k8s-pod-network.6680d3883d6150e75ffbd031f86c689a97a5be0f260c6442b2bb46b567c2ca40

secondary:

namespace: calico-apiserver

node: cadmin

pod: calico-apiserver-77dffffcdf-g2tcx

timestamp: 2021-09-30 09:46:57.45651816 +0000 UTC

- handle_id: k8s-pod-network.b10d7702bf334fc55a5e399a731ab3201ea9990a1e3bc79894abddd712646699

secondary:

namespace: calico-system

node: cadmin

pod: calico-kube-controllers-bdd5f97c5-554z5

timestamp: 2021-09-30 09:46:57.502351346 +0000 UTC

IPAMHandle

IPAMHandle用来记录IP分配的具体细节

apiVersion: crd.projectcalico.org/v1

kind: IPAMHandle

metadata:

name: k8s-pod-
network.8d75b941d85c4998016b72c83f9c5a75512c82c052357daf0ec8e67365635d93

spec:

block:

192.168.119.64/26: 1

deleted: false

handleID: k8s-pod-
network.8d75b941d85c4998016b72c83f9c5a75512c82c052357daf0ec8e67365635d93

创建Pod并查看IP配置情况



容器namespace

```
nsenter -t 720702 -n ip a
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
```

```
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

```
inet 127.0.0.1/8 scope host lo
```

```
valid_lft forever preferred_lft forever
```

```
3: eth0@if27: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UP group default
```

```
link/ether f2:86:d2:4f:1f:30 brd ff:ff:ff:ff:ff:ff link-netnsid 0
```

```
inet 192.168.119.84/32 brd 192.168.119.84 scope global eth0
```

```
valid_lft forever preferred_lft forever
```

```
nsenter -t 720702 -n ip r
```

```
default via 169.254.1.1 dev eth0
```

```
169.254.1.1 dev eth0 scope link
```

```
nsenter -t 720702 -n arp
```

Address	HWtype	HWaddress	Flags	Mask	Iface
169.254.1.1	ether	ee:ee:ee:ee:ee:ee	C		eth0
10.0.2.15	ether	ee:ee:ee:ee:ee:ee	C		eth0

主机Namespace

```
ip link
```

```
27: cali23a582ef038@if3:
```

```
<BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UP group default
```

```
link/ether ee:ee:ee:ee:ee:ee brd ff:ff:ff:ff:ff:ff link-netnsid 9
```

```
inet6 fe80::ecee:eeff:feee:eeee/64 scope link
```

```
valid_lft forever preferred_lft forever
```

```
ip route
```

```
192.168.119.84 dev cali23a582ef038 scope link
```

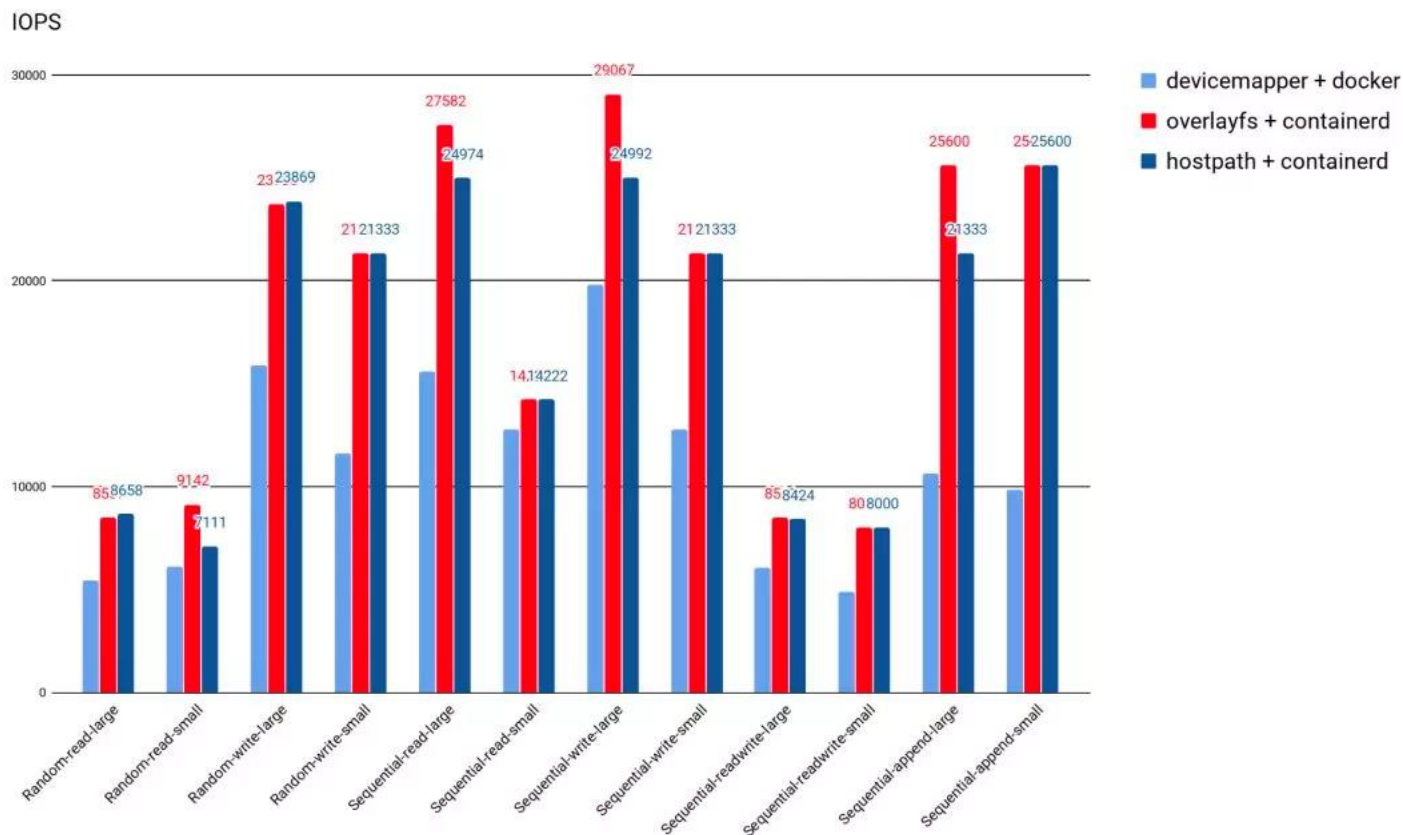

CNI plugin 的对比

解决方案	是否支持网络策略	是否支持 ipv6	基于网络层级	部署方式	命令行
Calico	是	是	L3 (IPinIP, BGP)	DaemonSet	calicoctl
Cilium	是	是	L3/L4 + L7 (filtering)	DaemonSet	cilium
Contiv	否	是	L2 (VxLan)/L3 (BGP)	DaemonSet	无
Flannel	否	否	L2 (VxLan)	DaemonSet	无
Weave net	是	是	L2 (VxLan)	DaemonSet	无

6. CSI

容器运行时存储

- 除外挂存储卷外，容器启动后，运行时所需文件系统性能直接影响容器性能；
- 早期的 Docker 采用 Device Mapper 作为容器运行时存储驱动，因为 OverlayFS 尚未合并进 Kernel；
- 目前 Docker 和 containerd 都默认以 OverlayFS 作为运行时存储驱动；
- OverlayFS 目前已经有非常好的性能，与 DeviceMapper 相比优 20%，与操作主机文件性能几乎一致。



存储卷插件管理

Kubernetes 支持以插件的形式来实现对不同存储的支持和扩展，这些扩展基于如下三种方式：

in-tree 插件

Kubernetes 社区已不再接受新的 in-tree 存储插件，新的存储必须通过 out-of-tree 插件进行支持

out-of-tree FlexVolume 插件

FlexVolume 是指 Kubernetes 通过调用计算节点的本地可执行文件与存储插件进行交互
FlexVolume 插件需要宿主机用 root 权限来安装插件驱动
FlexVolume 存储驱动需要宿主机安装 attach、mount 等工具，也需要具有 root 访问权限。

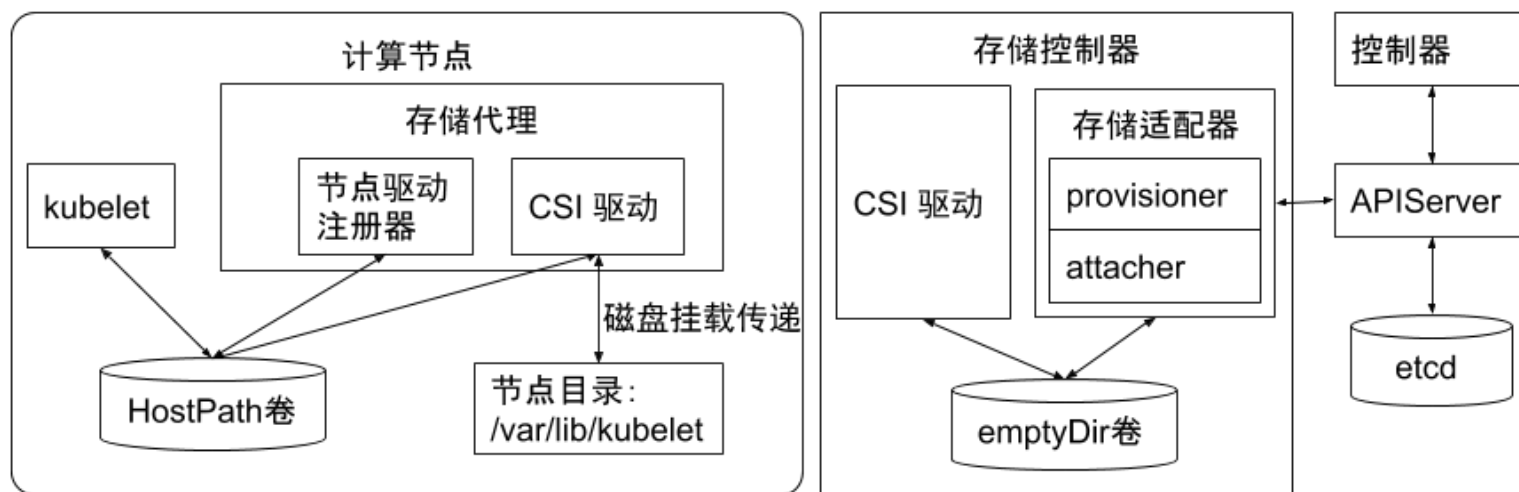
out-of-tree CSI 插件

out-of-tree CSI 插件

- CSI 通过 RPC 与存储驱动进行交互。
- 在设计 CSI 的时候，Kubernetes 对 CSI 存储驱动的打包和部署要求很少，主要定义了 Kubernetes 的两个相关模块：
 - **kube-controller-manager** :
 - kube-controller-manager 模块用于感知CSI驱动存在。
 - Kubernetes 的主控模块通过 Unix domain socket（而不是 CSI 驱动）或者其他方式进行直接地交互。
 - Kubernetes 的主控模块只与 Kubernetes 相关的 API 进行交互。
 - 因此 CSI 驱动若有依赖于 Kubernetes API 的操作，例如卷的创建、卷的 attach、卷的快照等，需要在 CSI 驱动里面通过 Kubernetes 的 API，来触发相关的 CSI 操作。
 - **kubelet**:
 - kubelet 模块用于与 CSI 驱动进行交互。
 - kubelet 通过 Unix domain socket 向 CSI 驱动发起 CSI 调用（如 NodeStageVolume、NodePublishVolume 等），再发起 mount 卷和 umount 卷。
 - kubelet 通过插件注册机制发现 CSI 驱动及用于和 CSI 驱动交互的 Unix Domain Socket。
 - 所有部署在 Kubernetes 集群中的 CSI 驱动都要通过 kubelet 的插件注册机制来注册自己。

CSI驱动

CSI 的驱动一般包含 external-attacher、external-provisioner、external-resizer、external-snapshotter、node-driver-register、CSI driver 等模块，可以根据实际的存储类型和需求进行不同方式的部署。



临时存储

常见的临时存储主要就是 emptyDir 卷。

emptyDir 是一种经常被用户使用的卷类型，顾名思义，“卷”最初是空的。当 Pod 从节点上删除时，emptyDir 卷中的数据也会被永久删除。但当 Pod 的容器因为某些原因退出再重启时，emptyDir 卷内的数据并不会丢失。

默认情况下，emptyDir 卷存储在支持该节点所使用的存储介质上，可以是本地磁盘或网络存储。emptyDir 也可以通过将 emptyDir.medium 字段设置为 “Memory” 来通知 Kubernetes 为容器安装 tmpfs，此时数据被存储在内存中，速度相对于本地存储和网络存储快很多。但是在节点重启的时候，内存数据会被清除；而如果存在磁盘上，则重启后数据依然存在。另外，使用 tmpfs 的内存也会计入容器的使用内存总量中，受系统的 Cgroup 限制。

emptyDir 设计的初衷主要是给应用充当缓存空间，或者存储中间数据，用于快速恢复。然而，这并不是说满足以上需求的用户都被推荐使用 emptyDir，我们要根据用户业务的实际特点来判断是否使用 emptyDir。因为 emptyDir 的空间位于系统根盘，被所有容器共享，所以在磁盘的使用率较高时会触发 Pod 的 eviction 操作，从而影响业务的稳定。

半持久化存储

常见的半持久化存储主要是hostPath卷。hostPath卷能将主机节点文件系统上的文件或目录挂载到指定 Pod 中。对普通用户而言一般不需要这样的卷，但是对很多需要获取节点系统信息的Pod而言，却是非常必要的。

例如，hostPath 的用法举例如下：

- 某个 Pod 需要获取节点上所有 Pod 的 log，可以通过 hostPath 访问所有 Pod 的 stdout 输出存储目录，例如 /var/log/pods 路径。
- 某个 Pod 需要统计系统相关的信息，可以通过 hostPath 访问系统的 /proc 目录。

使用 hostPath 的时候，除设置必需的 path 属性外，用户还可以有选择性地为 hostPath 卷指定类型，支持类型包含目录、字符设备、块设备等。

hostPath 卷需要注意

使用同一个目录的 Pod 可能会由于调度到不同的节点，导致目录中的内容有所不同。

Kubernetes 在调度时无法顾及由 hostPath 使用的资源。

Pod被删除后，如果没有特别处理，那么hostPath上写的数据会遗留到节点上，占用磁盘空间。

持久化存储

支持持久化的存储是所有分布式系统所必备的特性。针对持久化存储，Kubernetes 引入了 StorageClass、Volume、PVC（Persistent Volume Claim）、PV（Persistent Volume）的概念，将存储独立于 Pod 的生命周期来进行管理。

Kubernetes 目前支持的持久化存储包含各种主流的块存储和文件存储，譬如 awsElasticBlockStore、azureDisk、cinder、NFS、cephfs、iscsi 等，在大类上可以将其分为网络存储和本地存储两种类型。

StorageClass

StorageClass 用于指示存储的类型，不同的存储类型可以通过不同的 StorageClass 来为用户提供服务。StorageClass 主要包含存储插件 provisioner、卷的创建和 mount 参数等字段。

```
allowVolumeExpansion: true
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  annotations:
    storageclass.kubernetes.io/is-default-class: "false"
  name: rook-ceph-block
parameters:
  clusterID: rook-ceph
  csi.storage.k8s.io/controller-expand-secret-name: rook-csi-rbd-provisioner
  csi.storage.k8s.io/controller-expand-secret-namespace: rook-ceph
  csi.storage.k8s.io/fstype: ext4
  csi.storage.k8s.io/node-stage-secret-name: rook-csi-rbd-node
  csi.storage.k8s.io/node-stage-secret-namespace: rook-ceph
  csi.storage.k8s.io/provisioner-secret-name: rook-csi-rbd-provisioner
  csi.storage.k8s.io/provisioner-secret-namespace: rook-ceph
  imageFeatures: layering
  imageFormat: "2"
  pool: replicapool
provisioner: rook-ceph.rbd.csi.ceph.com
reclaimPolicy: Delete
volumeBindingMode: Immediate
```

PVC

由用户创建，代表用户对存储需求的声明，主要包含需要的存储大小、存储卷的访问模式、StorageClass 等类型，其中存储卷的访问模式必须与存储的类型一致

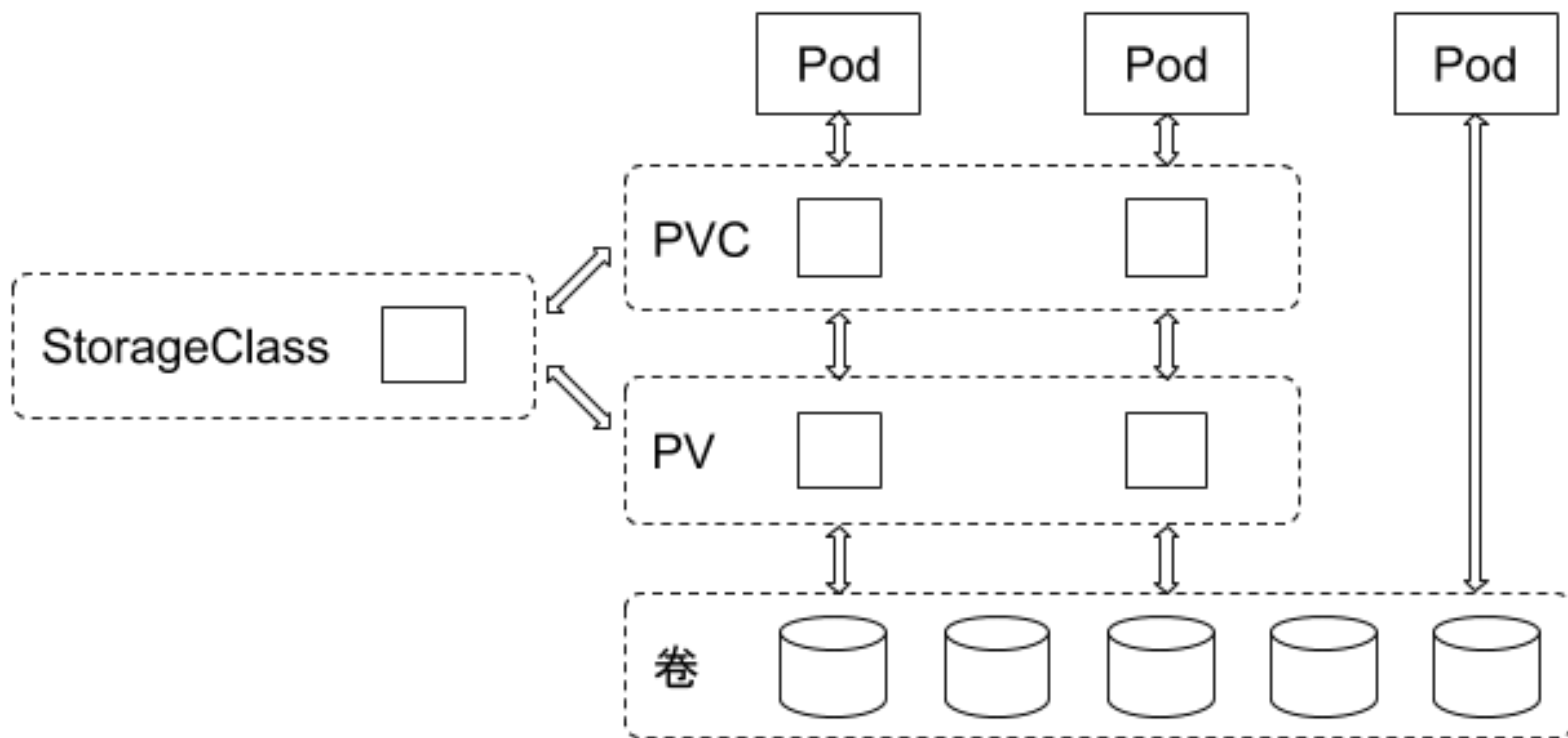
RWO	ReadWriteOnce	该卷只能在一个节点上被 mount，属性为可读可写
ROX	ReadOnlyMany	该卷可以在不同的节点上被 mount，属性为只读
RWX	ReadWriteMany	该卷可以在不同的节点上被 mount，属性为可读可写

PV

由集群管理员提前创建，或者根据 PVC 的申请需求动态地创建，它代表系统后端的真实的存储空间，可以称之为卷空间。

存储对象关系

用户通过创建 PVC 来申请存储。控制器通过 PVC 的 StorageClass 和请求的大小声明来存储后端创建卷，进而创建 PV，Pod 通过指定 PVC 来引用存储。



不同介质类型的磁盘，需要设置不同的 StorageClass，以便让用户做区分。StorageClass 需要设置磁盘介质的类型，以使用户了解该类存储的属性。

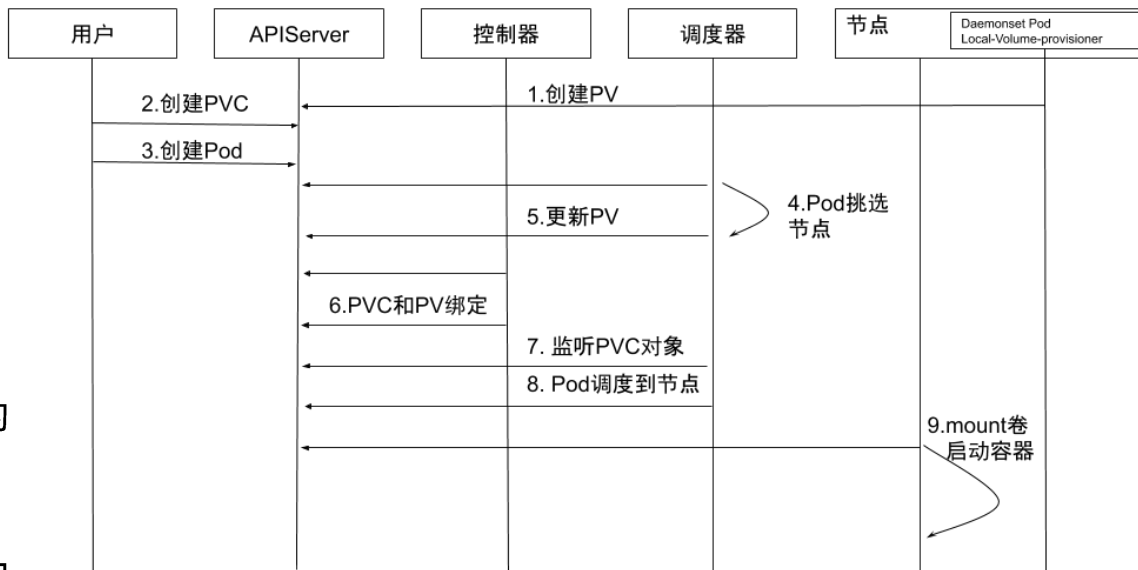
在本地存储的 PV 静态部署模式下，每个物理磁盘都尽量只创建一个 PV，而不是划分为多个分区来提供多个本地存储 PV，避免在使用时分区之间的 I/O 干扰。

本地存储需要配合磁盘检测来使用。当集群部署规模化后，每个集群的本地存储 PV 可能会超过几万个，如磁盘损坏将是频发事件。此时，需要在检测到磁盘损坏、丢盘等问题后，对节点的磁盘和相应的本地存储 PV 进行特定的处理，例如触发告警、自动 cordon 节点、自动通知用户等。

对于提供本地存储节点的磁盘管理，需要做到灵活管理和自动化。节点磁盘的信息可以归一、集中化管理。在 local-volume-provisioner 中增加部署逻辑，当容器运行起来时，拉取该节点需要提供本地存储的磁盘信息，例如磁盘的设备路径，以 Filesystem 或 Block 的模式提供本地存储，或者是否需要加入某个 LVM 的虚拟组（VG）等。local-volume-provisioner 根据获取的磁盘信息对磁盘进行格式化，或者加入到某个 VG，从而形成对本地存储支持的自动化闭环。

独占的 Local Volume

- **创建 PV**：通过local-volume-provisioner DaemonSet 创建本地存储的PV。
- **创建 PVC**：用户创建 PVC，由于它处于 pending 状态，所以 kube-controller-manager 并不会对该 PVC 做任何操作。
- **创建 Pod**：用户创建 Pod。
- **Pod 挑选节点**：kube-scheduler 开始调度Pod，通过 PVC 的 resources.request.storage 和 volumeMode 选择满足条件的 PV，并且为 Pod 选择一个合适的节点。
- **更新 PV**：kube-scheduler 将 PV 的 pv.Spec.claimRef 设置为对应的 PVC，并且设置 annotation pv.kubernetes.io/bound-by-controller 的值为 “yes”。
- **PVC 和 PV 绑定**：pv_controller 同步 PVC 和 PV 的状态，并将 PVC 和 PV 进行绑定。
- **监听 PVC 对象**：kube-scheduler 等待 PVC 的状态变成 Bound 状态。
- **Pod 调度到节点**：如果 PVC 的状态变为 Bound 则说明调度成功，而如果 PVC 一直处于 pending 状态，超时后会再次进行调度。
- **Mount 卷启动容器**：kubelet 监听到有 Pod 已经调度到节点上，对本地存储进行 mount 操作，并启动容器。



Dynamic Local Volume

CSI 驱动需要汇报节点上相关存储的资源信息，以便用于调度

但是机器的厂家不同，汇报方式也不同。

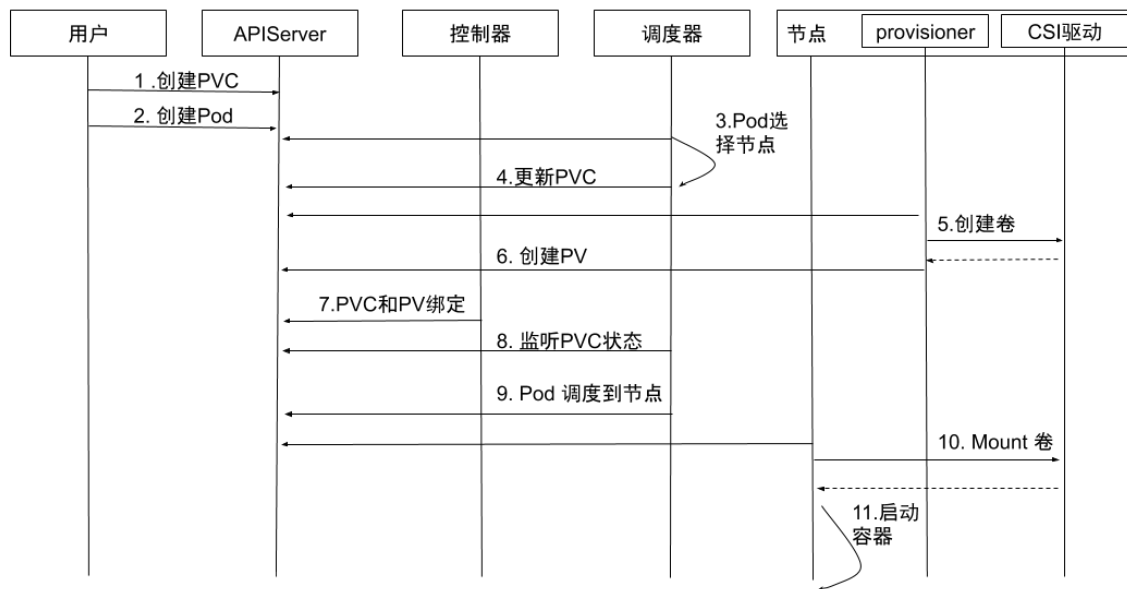
例如，有的厂家的机器节点上具有 NVMe、SSD、HDD 等多种存储介质，希望将这些存储介质分别进行汇报。

这种需求有别于其他存储类型的 CSI 驱动接口的需求，因此如何汇报节点的存储信息，以及如何让节点的存储信息应用于调度，目前并没有形成统一的意见。

集群管理员可以基于节点存储的实际情况对开源 CSI 驱动和调度进行一些代码修改，再进行部署和使用

Local Dynamic 的挂载流程

- **创建 PVC**: 用户创建 PVC, PVC 处于 pending 状态。
- **创建 Pod**: 用户创建 Pod。
- **Pod 选择节点**: kube-scheduler 开始调度 Pod, 通过 PVC 的 `pvc.spec.resources.request.storage` 等选择满足条件的节点。
- **更新 PVC**: 选择节点后, kube-scheduler 会给 PVC 添加包含节点信息的 annotation: `volume.kubernetes.io/selected-node: <节点名字>`。
- **创建卷**: 运行在节点上的容器 external-provisioner 监听到 PVC 带有该节点相关的 annotation, 向相应的 CSI 驱动申请分配卷。
- **创建PV**: PVC 申请到所需的存储空间后, external-provisioner 创建 PV, 该 PV 的 `pv.Spec.claimRef` 设置为对应的 PVC。
- **PVC和PV绑定**: kube-controller-manager 将 PVC 和 PV 进行绑定, 状态修改为 Bound。
- **监听PVC状态**: kube-scheduler 等待 PVC 变成 Bound 状态。
- **Pod调度到节点**: 当 PVC 的状态为 Bound 时, Pod 才算真正调度成功了。如果 PVC 一直处于 Pending 状态, 超时后会再次进行调度。
- **Mount卷**: kubelet 监听到有 Pod 已经调度到节点上, 对本地存储进行 mount 操作。
- **启动容器**: 启动容器。



Local Dynamic 的挑战

如果将磁盘空间作为一个存储池（例如 LVM）来动态分配，那么在分配出来的逻辑卷空间的使用上，可能会受到其他逻辑卷的 I/O 干扰，因为底层的物理卷可能是同一个。

如果 PV 后端的磁盘空间是一块独立的物理磁盘，则 I/O 就不会受到干扰。

不同介质类型的磁盘，需要设置不同的 StorageClass，以便让用户做区分。StorageClass 需要设置磁盘介质的类型，以使用户了解该类存储的属性。

在本地存储的 PV 静态部署模式下，每个物理磁盘都尽量只创建一个PV，而不是划分为多个分区来提供多个本地存储 PV，避免在使用时分区之间的 I/O 干扰。

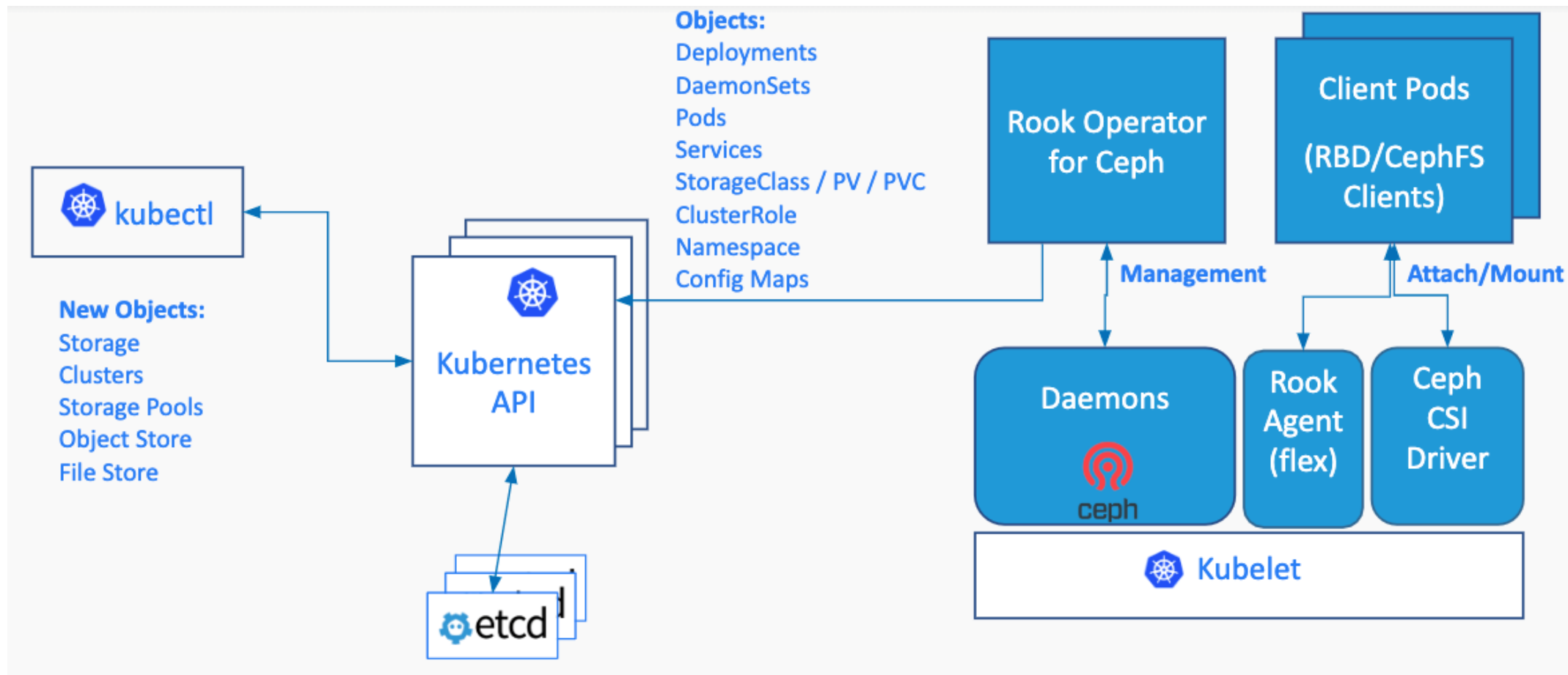
本地存储需要配合磁盘检测来使用。当集群部署规模化后，每个集群的本地存储 PV 可能会超过几万个，如磁盘损坏将是频发事件。此时，需要在检测到磁盘损坏、丢盘等问题后，对节点的磁盘和相应的本地存储 PV 进行特定的处理，例如触发告警、自动 cordon 节点、自动通知用户等。

对于提供本地存储节点的磁盘管理，需要做到灵活管理和自动化。节点磁盘的信息可以归一、集中化管理。在 local-volume-provisioner 中增加部署逻辑，当容器运行起来时，拉取该节点需要提供本地存储的磁盘信息，例如磁盘的设备路径，以 Filesystem 或 Block 的模式提供本地存储，或者是否需要加入某个 LVM 的虚拟组（VG）等。local-volume-provisioner 根据获取的磁盘信息对磁盘进行格式化，或者加入到某个 VG，从而形成对本地存储支持的自动化闭环。

Rook

Rook 是一款云原生环境下的开源分布式存储编排系统，目前支持 Ceph、NFS、EdgeFS、Cassandra、CockroachDB 等存储系统。它实现了一个自动管理的、自动扩容的、自动修复的分布式存储服务。Rook 支持自动部署、启动、配置、分配、扩容/缩容、升级、迁移、灾难恢复、监控以及资源管理。

Rook架构



Rook Operator

Rook Operator 是 Rook 的大脑，以 deployment 形式存在。

其利用 Kubernetes 的 controller-runtime 框架实现了 CRD，并进而接受 Kubernetes 创建资源的请求并创建相关资源（集群，pool，块存储服务，文件存储服务等）。

Rook Operator 监控存储守护进程，来确保存储集群的健康。

监听 Rook Discover 收集到的存储磁盘设备，并创建相应服务（Ceph 的话就是 OSD 了）。

Rook Discover

Rook Discover 是以 DaemonSet 形式部署在所有的存储机上的，其检测挂接到存储节点上的存储设备。把符合要求的存储设备记录下来，这样 Rook Operate 感知到以后就可以基于该存储设备创建相应服务了。

```
## discover device
$ lsblk --all --noheadings --list --output KNAME
$ lsblk /dev/vdd --bytes --nodeps --pairs --paths --output SIZE,ROTA,RO,TYPE,PKNAME,NAME,KNAME
$ udevadm info --query=property /dev/vdd
$ lsblk --noheadings --pairs /dev/vdd

## discover ceph inventory
$ ceph-volume inventory --format json
if device has ceph inv, device.CephVolumeData = CVData

## put device info into configmap per node
```


CSIDriver 发现

CSI 驱动发现：

如果一个 CSI 驱动创建 CSIDriver 对象, Kubernetes 用户可以通过 `get CSIDriver` 命令发现它们；

CSI 对象有如下特点：

- 自定义的 Kubernetes 逻辑；
- Kubernetes 对存储卷有一些列操作，这些 CSIDriver 可以自定义支持哪些操作？

Provisioner

CSI external-provisioner 是一个监控 Kubernetes PVC 对象的 Sidecar 容器。

当用户创建 PVC 后，Kubernetes 会监测 PVC 对应的 StorageClass，如果 StorageClass 中的 provisioner 与某插件匹配，该容器通过 CSI Endpoint（通常是 unix socket）调用 CreateVolume 方法。

如果 CreateVolume 方法调用成功，则 Provisioner sidecar 创建 Kubernetes PV 对象。

CSI External Provisioner

containers:

- args:

- `--csi-address=$(ADDRESS)`

- `--v=0`

- `--timeout=150s`

- `--retry-interval-start=500ms`

env:

- name: ADDRESS

value: `unix:///csi/csi-provisioner.sock`

image: `quay.io/k8scsi/csi-provisioner:v1.6.0`

name: `csi-provisioner`

resources: {}

volumeMounts:

- mountPath: `/csi`

name: `socket-dir`

- mountPath: `/var/run/secrets/kubernetes.io/serviceaccount`

name: `rook-csi-rbd-provisioner-sa-token-mxv84`

readOnly: `true`

CSI External Provisioner

- args:

- --nodeid=\$(NODE_ID)

- --endpoint=\$(CSI_ENDPOINT)

- --v=0

- --type=rbd

- --controllerserver=true

- --drivername=rook-ceph.rbd.csi.ceph.com

env:

- name: CSI_ENDPOINT

value: unix:///csi/csi-provisioner.sock

image: quay.io/cephcsi/cephcsi:v3.0.0

name: csi-rbdplugin

- emptyDir:

medium: Memory

name: socket-dir

Provisioner 代码

controller/controller.go

syncClaim -> provisionClaimOperation-> provisioner.Provision

pkg/operator/ceph/provisioner/provisioner.go

Provision-> createVolume

ceph.CreateImage

pkg/daemon/ceph/client/image.go

rbd create poolName/name -size sizeMB

volumeStore.StoreVolume

controller/volume_store.go

doSaveVolume

client.CoreV1().PersistentVolumes().Create(volume)

Provisioner log



```
I0816 10:17:32.535207    1 connection.go:153] Connecting to unix:///csi/csi-  
provisioner.sock  
I0816 10:17:54.361911    1 volume_store.go:97] Starting save volume queue  
I0816 10:17:54.461930    1 controller.go:1284] provision "default/mysql-pv-claim" class  
"rook-ceph-block": started  
I0816 10:17:54.462078    1 controller.go:848] Started provisioner controller rook-  
ceph.rbd.csi.ceph.com_csi-rbdplugin-provisioner-677577c77c-vwkzz_ca5971ab-2293-4e52-  
9bc9-c490f7f50b07!  
I0816 10:17:54.465752    1 event.go:281]  
Event(v1.ObjectReference{Kind:"PersistentVolumeClaim", Namespace:"default",  
Name:"mysql-pv-claim", UID:"24449707-6738-425c-ac88-de3c470cf91a", APIVersion:"v1",  
ResourceVersion:"45668", FieldPath:""}): type: 'Normal' reason: 'Provisioning' External  
provisioner is provisioning volume for claim "default/mysql-pv-claim"
```

Rook Agent



Rook Agent 是以 DaemonSet 形式部署在所有的存储机上的，其处理所有的存储操作，例如挂卸载存储卷以及格式化文件系统等。

CSI插件注册

spec:

hostNetwork: true

hostPID: true

- args:

- --v=0

- --csi-address=/csi/csi.sock

- --kubelet-registration-path=/var/lib/kubelet/plugins/rook-ceph.rbd.csi.ceph.com/csi.sock

image: quay.io/k8scsi/csi-node-driver-registrar:v1.2.0

name: driver-registrar

resources: {}

securityContext:

privileged: true

volumeMounts:

- mountPath: /csi

name: plugin-dir

- mountPath: /registration

name: registration-dir

CSI Driver

```
apiVersion: storage.k8s.io/v1
kind: CSIDriver
metadata:
  name: rook-ceph.rbd.csi.ceph.com
spec:
  attachRequired: true
  podInfoOnMount: false
  volumeLifecycleModes:
    - Persistent

ls /var/lib/kubelet/plugins/rook-
ceph.rbd.csi.ceph.com
csi.sock
```

CSI Driver

- args:
 - --nodeid=\$(NODE_ID)
 - --endpoint=\$(CSI_ENDPOINT)
 - --v=0
 - --type=rbd
 - --nodeserver=true
 - --drivername=rook-ceph.rbd.csi.ceph.com
 - --pidlimit=-1
 - --metricsport=9090
 - --metricspath=/metrics
 - --enablegrpcmetrics=true
- env:
 - name: CSI_ENDPOINT
 - value: unix:///csi/csi.sock
- image: quay.io/cephcsi/cephcsi:v3.0.0
- name: csi-rbdplugin
- securityContext:
 - allowPrivilegeEscalation: true
 - capabilities:
 - add:
 - SYS_ADMIN
 - privileged: true
- hostPath:
 - path: /var/lib/kubelet/plugins/rook-ceph.rbd.csi.ceph.com
 - type: DirectoryOrCreate
 - name: plugin-dir

Agent

```
pkg/daemon/ceph/agent/agent.go
```

```
flexvolume.NewController(a.context, volumeAttachmentController, volumeManager)
```

```
rpc.Register(flexvolumeController)
```

```
flexvolumeServer.Start
```

Cluster

针对不同 ceph cluster, rook 启动一组管理组件, 包括:

mon, mgr, osd, mds, rgw。

```
apiVersion: ceph.rook.io/v1
kind: CephCluster
metadata:
  name: rook-ceph
  namespace: rook-ceph
spec:
  cephVersion:
    image: ceph/ceph:v14.2.10
    dataDirHostPath: /var/lib/rook
  mon:
    count: 3
    allowMultiplePerNode: false
  mgr:
    modules:
      - name: pg_autoscaler
        enabled: true
  dashboard:
    enabled: true
  storage:
    useAllNodes: true
    useAllDevices: true
```

Pool

一个ceph cluster 可以有多个 pool,
定义副本数量, 故障域等多个属性。

```
apiVersion: ceph.rook.io/v1
kind: CephBlockPool
metadata:
  name: replicapool
  namespace: rook-ceph
spec:
  compressionMode: ""
  crushRoot: ""
  deviceClass: ""
  erasureCoded:
    algorithm: ""
    codingChunks: 0
    dataChunks: 0
  failureDomain: host
  replicated:
    requireSafeReplicaSize: false
    size: 1
    targetSizeRatio: 0
status:
  phase: Ready
```

Storage Class

StorageClass 是 Kubernetes 用来自动创建 PV 的对象。

```
allowVolumeExpansion: true
apiVersion: storage.k8s.io/v1
kind: StorageClass
  name: rook-ceph-block
parameters:
  clusterID: rook-ceph
  csi.storage.k8s.io/controller-expand-secret-name: rook-csi-rbd-provisioner
  csi.storage.k8s.io/controller-expand-secret-namespace: rook-ceph
  csi.storage.k8s.io/fstype: ext4
  csi.storage.k8s.io/node-stage-secret-name: rook-csi-rbd-node
  csi.storage.k8s.io/node-stage-secret-namespace: rook-ceph
  csi.storage.k8s.io/provisioner-secret-name: rook-csi-rbd-provisioner
  csi.storage.k8s.io/provisioner-secret-namespace: rook-ceph
  imageFeatures: layering
  imageFormat: "2"
  pool: replicapool
provisioner: rook-ceph.rbd.csi.ceph.com
reclaimPolicy: Delete
volumeBindingMode: Immediate
```

参考资料

<https://dramasamy.medium.com/life-of-a-packet-in-kubernetes-part-2-a07f5bf0ff14>

THANKS

 极客时间 | 训练营